

AND, OR, XOR

1. Overview

AND, **OR**, and **XOR** are the three most common bitwise operators. They work on the binary representation of integers and compare bits position by position.

Operator	C++	Python	Meaning
AND	<code>a & b</code>	<code>a & b</code>	Bit is 1 only if both bits are 1
OR	<code>a b</code>	<code>a b</code>	Bit is 1 if either bit is 1
XOR	<code>a ^ b</code>	<code>a ^ b</code>	Bit is 1 if bits are different

2. Intuition

Think of every integer as a row of switches. Each bit is either off **0** or on **1**.

- **AND** keeps only the switches that are on in both numbers.
- **OR** turns on every switch that is on in either number.
- **XOR** keeps only the switches where the two numbers disagree.

Example:

```
5 = 101
3 = 011

5 & 3 = 001 = 1
5 | 3 = 111 = 7
5 ^ 3 = 110 = 6
```

3. When to Use It

Use bitwise operators when:

- A problem mentions bits, binary, masks, flags, subsets, parity, or toggling.
- You need to store many boolean states compactly.
- You need fast set-like operations on small universes.
- You need XOR cancellation, such as finding a unique element.
- A problem asks for odd/even, power of two, hamming distance, or bit count.

Common trigger phrases:

- "bitwise AND/OR/XOR"
- "different bits"
- "appears twice except one"
- "mask"

- "toggle"
- "binary representation"

4. When Not to Use It

Do not force bitwise operators when:

- Normal arithmetic or a hash map is clearer and constraints are small.
- The values are negative and you are unsure about signed representation.
- The problem needs ordered data, not bit states.
- The universe size is large, for example arbitrary strings or IDs.
- Readability matters more than micro-optimization.

Common wrong assumption: XOR is not exponentiation. In C++ and Python, $\wedge$ means bitwise XOR, not power.

5. Core Concepts

Concept	Meaning	Why it matters
Bit position	Zero-indexed position from right	Used in masks and shifts
Mask	Number whose bits represent selected positions	Used to isolate or update bits
Idempotent	$x \& x = x, x \wedge x = 0$	$x = x \wedge 0$
XOR cancellation	$x \wedge x = 0, x \wedge 0 = x$	Core of single number problems
Parity	Odd/even count of 1s	XOR stores parity-like information

6. Step-by-Step Algorithm

For applying a bitwise operator:

1. Convert both numbers mentally to binary.
2. Align bits from the least significant bit.
3. Apply the operator to each pair of bits.
4. Convert the resulting binary number back to decimal if needed.

7. Dry Run

Input: $a = 12, b = 10$

```
12 = 1100
10 = 1010
```

Bit position	a bit	b bit	AND	OR	XOR
3	1	1	1	1	0
2	1	0	0	1	1

Bit position	a bit	b bit	AND	OR	XOR
1	0	1	0	1	1
0	0	0	0	0	0

Final:

```
12 & 10 = 1000 = 8
12 | 10 = 1110 = 14
12 ^ 10 = 0110 = 6
```

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a = 12, b = 10;

    cout << "AND: " << (a & b) << '\n';
    cout << "OR : " << (a | b) << '\n';
    cout << "XOR: " << (a ^ b) << '\n';

    return 0;
}
```

Example output:

```
AND: 8
OR : 14
XOR: 6
```

pYTHON IMPLEMENTATION

```
def bitwise_operations(a: int, b: int) -> tuple[int, int, int]:
    return a & b, a | b, a ^ b

a, b = 12, 10
and_value, or_value, xor_value = bitwise_operations(a, b)
print("AND:", and_value)
print("OR :", or_value)
print("XOR:", xor_value)
```

10. Code Explanation

The C++ code stores two integers and directly applies the three bitwise operators. Parentheses around expressions like `(a & b)` avoid precedence confusion while printing.

The Python function returns all three results as a tuple, making it easy to reuse in practice problems.

11. Complexity Analysis

Operation	Time	Space
<code>a & b</code>	$O(1)$	$O(1)$
<code>`a</code>	<code>b`</code>	$O(1)$
<code>a ^ b</code>	$O(1)$	$O(1)$

Technically the work depends on machine word size, but for standard integer constraints it is treated as constant time.

12. Common Patterns

Pattern	How to identify	Approach	Example problems
Mask checking	Need selected bits only	Use <code>x & mask</code>	LeetCode 191
Flag union	Combine active states	Use <code>`mask1</code>	<code>mask2`</code>
Difference bits	Need bits that differ	Use <code>a ^ b</code>	LeetCode 461 Hamming Distance
Cancellation	Pairs disappear	XOR all values	LeetCode 136 Single Number

13. Common Mistakes

- Using `^` for exponentiation.
- Forgetting parentheses: `a & b == 0` should be `(a & b) == 0`.
- Mixing logical operators `&&`, `||` with bitwise operators `&`, `|`.
- Applying bit tricks to negative numbers without considering representation.
- Assuming bit positions start from 1; in code they usually start from 0.

14. Edge Cases

- `a = 0, b = 0`
- One value is 0
- Both values are equal
- All bits set, such as `(1 << k) - 1`
- Large values near `INT_MAX`
- Negative values, if allowed by the problem

15. Variations

Variation	What changes	Usefulness
Bitwise NOT <code>~x</code>	Flips all bits	Useful but tricky with signed integers
Compound assignment	<code>x &= y</code> , <code>x ^= y</code>	<code>= y</code> , <code>x ^= y</code>
Long long masks	Use <code>1LL << i</code>	Important when <code>i >= 31</code>

16. Related Algorithms/Data Structures

- Bitmasking: Uses these operators to represent sets.
- XOR basis: Uses XOR to represent linear independence over bits.
- DP with bitmask: Uses masks as states.
- Bitset optimization: Uses word-level bitwise operations to speed up boolean DP.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Number of 1 Bits	LeetCode	AND and bit count	Easy
Hamming Distance	LeetCode	XOR different bits	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Bitwise AND of Numbers Range	LeetCode	Common prefix bits	Medium
Single Number III	LeetCode	XOR partition	Medium
XOR Queries of a Subarray	LeetCode	Prefix XOR	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Maximum XOR With an Element From Array	LeetCode	Trie and XOR	Hard
XOR on Segment	Codeforces	Lazy segment tree with bits	Hard
Xor of 3	CodeChef/CP	XOR properties	Hard

18. Interview Explanation

"Bitwise AND, OR, and XOR operate on individual bits. AND keeps common set bits, OR combines set bits, and XOR keeps positions where bits differ. XOR is especially useful because equal values cancel out, which helps in many unique-element and parity problems."

19. Revision Notes

- $a \& b$: common set bits.
- $a \mid b$: union of set bits.
- $a \wedge b$: differing bits.
- $x \wedge x = 0, x \wedge 0 = x$.
- Use parentheses in comparisons.
- Use `1LL << i` in C++ for large shifts.

20. Final Cheat Sheet

Need	Code
Common bits	$a \& b$
Combine bits	<code>`a</code>
Different bits	$a \wedge b$
Clear selected bits	$x \& \sim \text{mask}$
Toggle selected bits	$x \wedge \text{mask}$

Left Shift, Right Shift

1. Overview

Left shift and right shift move bits left or right.

Operation	Code	Meaning for non-negative integers
Left shift	$x \ll k$	Multiply by 2^k
Right shift	$x \gg k$	Divide by 2^k , floor for non-negative integers

2. Intuition

In decimal, appending a zero multiplies by 10. In binary, shifting left appends zero bits and multiplies by powers of 2.

```
5 = 101
5 << 1 = 1010 = 10
5 << 2 = 10100 = 20
```

Right shift removes bits from the right.

```
20 = 10100
20 >> 2 = 101 = 5
```

3. When to Use It

Use shifts when:

- You need 2^k .
- You need to build masks like $1 \ll i$.
- You need to inspect, set, clear, or toggle the i th bit.
- You need fast multiply/divide by powers of two for non-negative integers.
- You are enumerating subsets from 0 to $(1 \ll n) - 1$.

Trigger phrases:

- "ith bit"
- "2 to the power"
- "mask"
- "subset"
- "binary position"

4. When Not to Use It

Avoid shifts when:

- The shift count can be negative.
- The shift count is greater than or equal to the bit width in C++.
- You are shifting signed negative integers and need portable behavior.
- Multiplication or division is clearer and performance is irrelevant.

5. Core Concepts

Concept	Meaning	Example
$1 \ll i$	Number with only bit i set	$1 \ll 3 = 8$
Mask width	Number of bits available	<code>int</code> often has 32 bits
Overflow	Shifted value may exceed type range	Use $1LL \ll i$
Arithmetic right shift	Keeps sign bit for negatives on many systems	Avoid relying on it in interviews

6. Step-by-Step Algorithm

To create a mask for bit i :

1. Start with binary 1 .
2. Shift it left by i .
3. Use the result as the mask.

To extract bits by shifting:

1. Move the desired bit to the rightmost position.
2. AND with 1 .
3. The result is 0 or 1 .

7. Dry Run

Input: $x = 22, i = 2$

```
22 = 10110
1 << 2 = 00100
```

Step	Expression	Binary	Decimal
Original	x	10110	22
Mask	$1 \ll 2$	00100	4
Extract	$(x \gg 2) \& 1$	1	1

Final: bit 2 is set.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long powerOfTwo(int k) {
    return 1LL << k;
}

int getBit(long long x, int i) {
    return (x >> i) & 1LL;
}

int main() {
    long long x = 22;
    int i = 2;

    cout << "2^5 = " << powerOfTwo(5) << '\n';
    cout << "Bit " << i << " of " << x << " = " << getBit(x, i) << '\n';

    return 0;
}
```

pYTHON IMPLEMENTATION

```
def power_of_two(k: int) -> int:
    return 1 << k

def get_bit(x: int, i: int) -> int:
    return (x >> i) & 1
```

```
print(power_of_two(5))
print(get_bit(22, 2))
```

10. Code Explanation

`powerOfTwo` uses `1LL << k` so the calculation is done using `long long`. `getBit` shifts the target bit to position `0` and masks it with `1`.

Python integers have arbitrary precision, so large shifts are safer in Python, but very large shifts can still create huge numbers.

11. Complexity Analysis

Operation	Time	Space
Left shift	$O(1)$	$O(1)$
Right shift	$O(1)$	$O(1)$
Get bit	$O(1)$	$O(1)$

For extremely large Python integers, complexity depends on the number of machine words.

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Power of two masks	Need 2^n states	<code>1 << n</code>	Subsets
Bit extraction	Need <i>i</i> th bit	<code>(x >> i) & 1</code>	Binary watch
Range compression	Small $n \leq 20$	Use masks from <code>0</code> to <code>1 << n</code>	TSP DP

13. Common Mistakes

- Using `1 << 31` with signed `int`.
- Forgetting that bit positions are zero-indexed.
- Writing `1 << n - 1` without knowing precedence; prefer `1 << (n - 1)`.
- Shifting by a value outside valid range in C++.
- Assuming right shift of negative numbers behaves the same everywhere.

14. Edge Cases

- `k = 0`
- `i = 0`
- Large `i`, such as `i = 60`
- `x = 0`
- Negative `x`, if the problem allows it

15. Variations

Variation	Use
<code>1LL << i</code>	64-bit masks
<code>1ULL << i</code>	Unsigned masks
<code>x >>= 1</code> loop	Process bits one by one
<code>mask << 1</code> transitions	DP over shifted states

16. Related Algorithms/Data Structures

- Bitmasking uses shifts to create masks.
- Binary exponentiation uses shifting to inspect exponent bits.
- Fenwick tree uses lowbit, which is based on bit representation.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Convert Binary Number in a Linked List to Integer	LeetCode	Left shift accumulation	Easy
Power of Two	LeetCode	Shift/mask check	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Subsets	LeetCode	<code>1 << n</code> masks	Medium
Binary Watch	LeetCode	Bit counts and shifts	Medium
Counting Bits	LeetCode	Shift recurrence	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Shortest Path Visiting All Nodes	LeetCode	Bitmask states	Hard
Count Number of Maximum Bitwise-OR Subsets	LeetCode	Mask enumeration	Hard
Hamiltonian Flights	CSES	Bitmask DP	Hard

18. Interview Explanation

"Left shift moves bits left and is equivalent to multiplying a non-negative integer by powers of two. Right shift moves bits right and is equivalent to integer division by powers of two. They are most useful for creating masks and extracting specific bits."

19. Revision Notes

- $1 \ll i$ creates a mask for bit i .
- Use $1LL \ll i$ when i can be 31 or more.
- $(x \gg i) \& 1$ extracts bit i .
- Avoid shifting signed negatives in portable code.

20. Final Cheat Sheet

Need	Code
2^k	$1LL \ll k$
Get bit	$(x \gg i) \& 1$
Number of masks	$1 \ll n$
Divide non-negative by 2^k	$x \gg k$

Check if Bit is Set

1. Overview

Checking if a bit is set means testing whether the i th bit of a number is 1 .

2. Intuition

Create a mask with only the i th bit set. If x also has that bit set, $x \& \text{mask}$ will be non-zero.

```
x = 13 = 1101
i = 2
mask = 0100
x & mask = 0100, so bit 2 is set
```

3. When to Use It

Use this when:

- You need to check if an element is included in a subset mask.
- You are iterating over chosen items.
- A problem asks whether a particular bit is 0 or 1 .
- You need state transitions in bitmask DP.

Trigger phrases:

- "included in subset"
- "ith bit"
- "is selected"
- "is flag enabled"

4. When Not to Use It

Do not use it when:

- The state is better represented by a boolean array because indices are large or sparse.
- The bit position may exceed the integer width.
- You need count or order of selected items, not just membership.

5. Core Concepts

Concept	Meaning
Mask	<code>1LL << i</code>
Test by AND	<code>(x & mask) != 0</code>
Extract as 0/1	<code>(x >> i) & 1</code>
Zero-indexing	Rightmost bit is bit <code>0</code>

6. Step-by-Step Algorithm

1. Read `x` and bit position `i`.
2. Compute `mask = 1LL << i`.
3. Compute `x & mask`.
4. If result is non-zero, the bit is set.
5. Otherwise, the bit is not set.

7. Dry Run

Input: `x = 18, i = 4`

```
18 = 10010
1 << 4 = 10000
```

Step	Value
<code>x</code>	<code>10010</code>
<code>mask</code>	<code>10000</code>
<code>x & mask</code>	<code>10000</code>
Answer	Set

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```
bool isBitSet(long long x, int i) {
    return (x & (1LL << i)) != 0;
}

int main() {
    long long x = 18;
    int i = 4;

    cout << (isBitSet(x, i) ? "Set" : "Not set") << '\n';
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def is_bit_set(x: int, i: int) -> bool:
    return (x & (1 << i)) != 0

print("Set" if is_bit_set(18, 4) else "Not set")
```

10. Code Explanation

The function builds a one-bit mask using `1LL << i`. The expression `x & mask` keeps only bit `i`. If the result is not zero, that bit was originally `1`.

11. Complexity Analysis

Operation	Time	Space
Build mask	$O(1)$	$O(1)$
Check bit	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Subset membership	Is item <code>i</code> selected?	<code>mask & (1 << i)</code>	LeetCode Subsets
DP transition	Add unused item	Check bit before transition	CSES Hamiltonian Flights
Flags	Permission enabled?	AND with flag mask	System design style questions

13. Common Mistakes

- Writing `(x & (1 << i)) == 1`; this only works for `i = 0`.
- Using 1-based bit positions when code expects 0-based.
- Using `int` shift for large `i`.
- Missing parentheses in conditions.

14. Edge Cases

- $x = 0$
- $i = 0$
- Highest allowed bit
- Negative x
- i outside allowed range

15. Variations

Variation	Code	Use
Boolean check	<code>(x & (1LL << i)) != 0</code>	Conditions
Extract bit	<code>(x >> i) & 1</code>	Need exact 0/1
Check unset	<code>(x & (1LL << i)) == 0</code>	Add item if missing

16. Related Algorithms/Data Structures

- Set bit, clear bit, toggle bit are update operations after checking.
- Bitmask DP checks whether an item is already used.
- Subset generation checks each bit to construct a subset.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Binary Number with Alternating Bits	LeetCode	Check adjacent bits	Easy
Number of 1 Bits	LeetCode	Inspect bits	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Subsets	LeetCode	Check included elements	Medium
Maximum Product of Word Lengths	LeetCode	Mask membership	Medium
Matchsticks to Square	LeetCode	Used-state mask	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Shortest Path Visiting All Nodes	LeetCode	Visited mask	Hard
Smallest Sufficient Team	LeetCode	Skill mask	Hard
Hamiltonian Flights	CSES	Bitmask DP	Hard

18. Interview Explanation

"To check if the i th bit is set, I create a mask with only that bit on using $1 \ll i$, then AND it with the number. If the result is non-zero, the bit exists in the number."

19. Revision Notes

- Correct check: $(x \& (1LL \ll i)) \neq 0$.
- Do not compare result to 1 unless checking bit 0 .
- Rightmost bit has index 0 .
- Use `long long` for high bit positions.

20. Final Cheat Sheet

Need	Code
Is bit set?	$(x \& (1LL \ll i)) \neq 0$
Is bit unset?	$(x \& (1LL \ll i)) == 0$
Extract bit	$(x \gg i) \& 1$

Set Bit

1. Overview

Setting a bit means making the i th bit of a number equal to 1 , without changing other bits.

2. Intuition

OR with a mask. Since $0 \mid 1 = 1$ and $1 \mid 1 = 1$, the target bit becomes 1 , while all other bits stay unchanged because they are ORed with 0 .

3. When to Use It

Use set bit when:

- Adding an item to a subset mask.
- Marking a state as visited.
- Enabling a flag.
- Building masks from input.

Trigger phrases:

- "mark selected"
- "add this element"
- "enable flag"
- "turn on bit"

4. When Not to Use It

Avoid it when:

- You need to append to a dynamic set of large values; use `unordered_set`.
- The position can exceed mask width.
- You need counts of duplicates; a single bit only stores yes/no.

5. Core Concepts

Concept	Explanation
OR update	<code>`x</code>
Idempotent	Setting an already set bit changes nothing
Assignment	Use <code>`x = x</code>

6. Step-by-Step Algorithm

1. Choose the bit position `i`.
2. Build `mask = 1LL << i`.
3. Compute `x = x | mask`.
4. Return or store updated `x`.

7. Dry Run

Input: `x = 8, i = 1`

```
x      = 1000
mask   = 0010
OR     = 1010 = 10
```

Step	Value
Initial <code>x</code>	8
Mask	2
Updated <code>x</code>	10

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long setBit(long long x, int i) {
    return x | (1LL << i);
}

int main() {
    cout << setBit(8, 1) << '\n';
}
```

```
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def set_bit(x: int, i: int) -> int:
    return x | (1 << i)

print(set_bit(8, 1))
```

10. Code Explanation

`1 << i` creates a mask with only bit `i` set. OR copies all existing `1` bits from `x` and also turns on bit `i`.

11. Complexity Analysis

Operation	Time	Space
Set one bit	$O(1)$	$O(1)$
Set several bits with one mask	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Add item to mask	Include element <code>i</code>	<code>`mask</code>	<code>(1 < i)`</code>
Visit node	State includes node	<code>`visited</code>	<code>(1 < node)`</code>
Skill coverage	Add person skills	<code>`teamMask</code>	<code>personMask`</code>

13. Common Mistakes

- Forgetting assignment: `x | (1 << i)` does not modify `x` unless stored.
- Using XOR to set; XOR toggles and may turn the bit off.
- Shifting by a 1-based index accidentally.

14. Edge Cases

- Bit already set.
- `i = 0`.
- Large `i`.
- `x = 0`.
- Multiple set operations on the same bit.

15. Variations

Variation	Code
Set one bit	<code>`x</code>
Set many bits	<code>`x</code>
Add subset	<code>`state</code>

16. Related Algorithms/Data Structures

- Check bit before setting to avoid duplicate transitions.
- Toggle bit differs because it flips existing value.
- Bitmask DP uses set bit to move from one state to a larger state.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Subsets	LeetCode	Add elements to mask	Easy/Medium
Binary Watch	LeetCode	Build bit states	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Smallest Sufficient Team	LeetCode	OR skill masks	Medium/Hard
Maximum Product of Word Lengths	LeetCode	Build character masks	Medium
Partition to K Equal Sum Subsets	LeetCode	Used element mask	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Shortest Path Visiting All Nodes	LeetCode	Set visited node	Hard
Hamiltonian Flights	CSES	Add city to path mask	Hard
Close Group	AtCoder	Build subset masks	Hard

18. Interview Explanation

"To set a bit, I OR the number with a mask containing only that bit. OR guarantees that the target bit becomes 1, while the remaining bits stay the same."

19. Revision Notes

- Set bit formula: $x \mid (1LL \ll i)$.
- In-place: $x \mid= (1LL \ll i)$.

- Setting twice has no extra effect.
- Use OR, not XOR.

20. Final Cheat Sheet

Need	Code
Set bit <i>i</i>	<code>`x</code>
Add item to subset	<code>`mask</code>
Enable flags	<code>`flags</code>

Clear Bit

1. Overview

Clearing a bit means making the *i*th bit equal to 0, without changing other bits.

2. Intuition

Create a mask with bit *i* as 1, invert it so only bit *i* becomes 0, then AND with the number.

```
x      = 1110
mask   = 0100
~mask  = 1011
result = 1010
```

3. When to Use It

Use clear bit when:

- Removing an item from a subset mask.
- Disabling a flag.
- Backtracking with bitmasks.
- Marking a resource as unavailable.

Trigger phrases:

- "remove from mask"
- "turn off bit"
- "disable flag"
- "unselect"

4. When Not to Use It

Avoid it when:

- You need to remove multiple counted occurrences; bits cannot store frequency.

- The universe is too large for one integer mask.
- The bit position is invalid.

5. Core Concepts

Concept	Explanation
NOT mask	$\sim(1LL \ll i)$ has all 1s except bit i
AND clearing	$x \& \sim\text{mask}$ preserves all bits except target
Idempotent	Clearing an already clear bit changes nothing

6. Step-by-Step Algorithm

1. Build $\text{mask} = 1LL \ll i$.
2. Invert mask: $\text{clearMask} = \sim\text{mask}$.
3. Compute $x = x \& \text{clearMask}$.
4. Return updated value.

7. Dry Run

Input: $x = 15, i = 2$

Step	Binary	Decimal
x	1111	15
$1 \ll 2$	0100	4
After clear	1011	11

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long clearBit(long long x, int i) {
    return x & ~(1LL << i);
}

int main() {
    cout << clearBit(15, 2) << '\n';
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def clear_bit(x: int, i: int) -> int:
    return x & ~(1 << i)

print(clear_bit(15, 2))
```

10. Code Explanation

The mask `1LL << i` isolates the target bit. `~` flips the mask, making the target bit `0` and other bits `1`. AND with this inverted mask forces the target bit to `0`.

11. Complexity Analysis

Operation	Time	Space
Clear one bit	$O(1)$	$O(1)$
Clear mask	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Remove selected item	Need mask without item <code>i</code>	<code>mask & ~(1 << i)</code>	Backtracking masks
Iterate submasks	Remove bits repeatedly	<code>(sub - 1) & mask</code>	SOS DP
Disable flag	Turn off permission	<code>flags &= ~flag</code>	Flag systems

13. Common Mistakes

- Writing `x & (1 << i)`, which checks instead of clears.
- Forgetting parentheses around `1LL << i`.
- Confusing clear with toggle.
- Problems with `~` and signed integers when printing binary.

14. Edge Cases

- Bit already clear.
- Clear bit `0`.
- Clear highest bit.
- `x = 0`.
- Negative `x`, if allowed.

15. Variations

Variation	Code
Clear one bit	<code>x &= ~(1LL << i)</code>

Variation	Code
Clear all bits in mask	<code>x &= ~mask</code>
Keep lower <code>k</code> bits only	<code>x &= (1LL << k) - 1</code>

16. Related Algorithms/Data Structures

- Toggle bit flips instead of forcing 0.
- Brian Kernighan clears the lowest set bit using `x &= x - 1`.
- DP transitions may remove items from a state.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Number of 1 Bits	LeetCode	Clear lowest set bit	Easy
Power of Two	LeetCode	Clear lowest set bit	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Subsets II	LeetCode	Mask/backtracking	Medium
Partition to K Equal Sum Subsets	LeetCode	Used mask updates	Medium
Maximum Product of Word Lengths	LeetCode	Character masks	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
SOS DP Problems	Codeforces	Remove bits/submasks	Hard
Hamiltonian Flights	CSES	Previous mask	Hard
Close Group	AtCoder	Subset transitions	Hard

18. Interview Explanation

"To clear bit `i`, I AND the number with the inverse of `1 << i`. The inverse mask has a zero only at the target bit, so AND forces that bit to zero and leaves the rest unchanged."

19. Revision Notes

- Clear bit: `x & ~(1LL << i)`.
- In-place: `x &= ~(1LL << i)`.
- `x & (x - 1)` clears the lowest set bit.

- Be careful with $\sim$ on signed values.

20. Final Cheat Sheet

Need	Code
Clear bit i	$x \&= \sim(1LL \ll i)$
Remove item	$mask \& \sim(1 \ll i)$
Clear lowest set bit	$x \&= x - 1$

Toggle Bit

1. Overview

Toggling a bit means flipping it: 0 becomes 1 , and 1 becomes 0 .

2. Intuition

XOR with 1 flips a bit. XOR with 0 leaves a bit unchanged. So XOR with a one-bit mask flips only the target bit.

3. When to Use It

Use toggle bit when:

- A state changes between on/off.
- You need parity of occurrences.
- You are solving problems where pairs cancel.
- You need to flip membership in a set.

Trigger phrases:

- "toggle"
- "flip"
- "odd/even occurrence"
- "switch state"

4. When Not to Use It

Avoid toggle when:

- You specifically need to force a bit to 1 ; use set bit.
- You specifically need to force a bit to 0 ; use clear bit.
- Repeated operations may accidentally undo your update.

5. Core Concepts

Concept	Meaning
XOR flip	$b \oplus 1$ flips bit b
XOR unchanged	$b \oplus 0 = b$
Toggle twice	Returns to original
Parity	XOR stores odd/even count behavior

6. Step-by-Step Algorithm

1. Build $mask = 1LL \ll i$.
2. Compute $x = x \oplus mask$.
3. The i th bit is flipped.
4. Other bits remain unchanged.

7. Dry Run

Input: $x = 10, i = 1$

```
x      = 1010
mask   = 0010
x^m    = 1000 = 8
```

Toggle again:

```
1000 ^ 0010 = 1010 = 10
```

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long toggleBit(long long x, int i) {
    return x ^ (1LL << i);
}

int main() {
    cout << toggleBit(10, 1) << '\n';
    cout << toggleBit(8, 1) << '\n';
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def toggle_bit(x: int, i: int) -> int:
    return x ^ (1 << i)

print(toggle_bit(10, 1))
print(toggle_bit(8, 1))
```

10. Code Explanation

`1 << i` creates a mask with one set bit. XOR flips exactly that bit because target bit is XORed with `1`, while all other bits are XORed with `0`.

11. Complexity Analysis

Operation	Time	Space
Toggle one bit	$O(1)$	$O(1)$
Toggle mask	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Parity mask	Characters with odd counts	Toggle char bit	LeetCode 1371
Unique element	Duplicate values cancel	XOR all values	LeetCode 136
Switch membership	Add/remove same operation	<code>mask ^= bit</code>	Backtracking

13. Common Mistakes

- Using toggle when the operation should be idempotent.
- Forgetting that toggling twice cancels out.
- Comparing `x ^ mask` without assigning when an update is needed.
- Using addition/subtraction instead of bit operations for masks.

14. Edge Cases

- Bit currently `0`.
- Bit currently `1`.
- Toggle same bit multiple times.
- `x = 0`.
- High bit toggles.

15. Variations

Variation	Code
Toggle bit <code>i</code>	<code>x ^= (1LL << i)</code>
Toggle several bits	<code>x ^= mask</code>
Toggle lowercase char parity	<code>mask ^= 1 << (c - 'a')</code>

16. Related Algorithms/Data Structures

- XOR properties explain why toggling works.
- Single number problems use XOR cancellation.
- Parity masks use toggling for character frequencies.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Single Number	LeetCode	XOR cancellation	Easy
Hamming Distance	LeetCode	XOR different bits	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Find the Longest Substring Containing Vowels in Even Counts	LeetCode	Parity mask	Medium
Single Number III	LeetCode	XOR partition	Medium
Wonderful Substrings	LeetCode	Toggle char parity	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Count The Repetitions of Parity States	Codeforces	Parity masks	Hard
Palindrome Paths in Tree	LeetCode	Path parity mask	Hard
XOR Basis Problems	Codeforces	XOR state changes	Hard

18. Interview Explanation

"To toggle a bit, I XOR the number with a mask containing that bit. XOR with 1 flips a bit, and XOR with 0 keeps it unchanged."

19. Revision Notes

- Toggle formula: $x \oplus= (1LL \ll i)$.
- Toggle twice returns original.
- Use for parity, not for forced set/clear.
- XOR is the key operator.

20. Final Cheat Sheet

Need	Code
Toggle bit	$x \oplus= (1LL \ll i)$
Toggle mask	$x \oplus= mask$
Toggle character parity	$mask \oplus= 1 \ll id$

Count Set Bits

1. Overview

Counting set bits means finding how many **1** bits are present in the binary representation of a number. It is also called population count or Hamming weight.

2. Intuition

Each **1** bit represents an active flag or selected item. Counting set bits tells us how many items are selected.

Two common methods:

- Check every bit one by one.
- Repeatedly remove the lowest set bit using Brian Kernighan's algorithm.

3. When to Use It

Use it when:

- Need size of a subset mask.
- Need Hamming weight.
- Need to compare binary density.
- Need parity or number of selected elements.
- DP depends on subset size.

Trigger phrases:

- "number of set bits"
- "Hamming weight"
- "selected count"
- "population count"

4. When Not to Use It

Avoid manual counting when:

- Built-in functions are allowed and clearer.
- You count bits repeatedly for many numbers up to `n`; use DP precomputation.
- The number can be negative and the bit width is unspecified.

5. Core Concepts

Concept	Meaning
Popcount	Number of <code>1</code> bits
Hamming weight	Same as popcount
<code>__builtin_popcount</code>	C++ built-in for <code>int</code>
<code>__builtin_popcountll</code>	C++ built-in for <code>long long</code>
<code>int.bit_count()</code>	Python built-in

6. Step-by-Step Algorithm

Brian Kernighan method:

1. Initialize `count = 0`.
2. While `x > 0`:
3. Replace `x` with `x & (x - 1)`.
4. Increment `count`.
5. Return `count`.

Why it works: `x & (x - 1)` removes the lowest set bit.

7. Dry Run

Input: `x = 13`

```
13 = 1101
```

Iteration	x before	x after <code>x & (x - 1)</code>	Count
1	1101	1100	1
2	1100	1000	2
3	1000	0000	3

Final answer: `3`

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

int countSetBits(long long x) {
    int count = 0;
    while (x > 0) {
        x &= (x - 1); // remove lowest set bit
        count++;
    }
    return count;
}

int main() {
    long long x = 13;
    cout << countSetBits(x) << '\n';
    cout << __builtin_popcountll(x) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def count_set_bits(x: int) -> int:
    count = 0
    while x > 0:
        x &= x - 1
        count += 1
    return count

print(count_set_bits(13))
print((13).bit_count())

```

10. Code Explanation

The loop runs once per set bit, not once per total bit position. Each iteration removes exactly one **1** bit, so the count after the loop equals the number of set bits.

The built-in version is preferred in contests unless the problem asks you to implement manually.

11. Complexity Analysis

Method	Time	Space
Check all bits	$O(\log x)$	$O(1)$
Brian Kernighan	$O(\text{number of set bits})$	$O(1)$
Built-in popcount	$O(1)$ on fixed-width integers	$O(1)$

Method	Time	Space
Precompute up to n	$O(n)$	$O(n)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Subset size	Need cardinality of mask	<code>popcount(mask)</code>	Bitmask DP
Hamming distance	Different bit count	<code>popcount(a ^ b)</code>	LeetCode 461
Sort by bits	Compare popcount	Built-in popcount	LeetCode 1356

13. Common Mistakes

- Using `__builtin_popcount` for `long long`; use `__builtin_popcountll`.
- Infinite loop with negative numbers.
- Forgetting to update `x` inside loop.
- Counting bits by converting to string in performance-critical code.

14. Edge Cases

- `x = 0`, answer `0`.
- `x = 1`, answer `1`.
- All bits set.
- Large 64-bit values.
- Negative values, if problem defines fixed width.

15. Variations

Variation	Use
Precompute <code>bits[i] = bits[i >> 1] + (i & 1)</code>	Many queries
<code>popcount(a ^ b)</code>	Hamming distance
<code>popcount(mask) == k</code>	Subsets of fixed size

16. Related Algorithms/Data Structures

- Brian Kernighan algorithm is the classic manual method.
- Subsets using bitmask often uses popcount for filtering.
- Bitset optimization can count set bits using `.count()`.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Number of 1 Bits	LeetCode	Popcount	Easy
Counting Bits	LeetCode	DP popcount	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Sort Integers by The Number of 1 Bits	LeetCode	Sort by popcount	Medium
Total Hamming Distance	LeetCode	Count set bits per position	Medium
Beautiful Arrangement	LeetCode	Mask size DP	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Minimum Incompatibility	LeetCode	Fixed-size subsets	Hard
Can I Win	LeetCode	State masks	Hard
Hamiltonian Flights	CSES	Mask DP by size	Hard

18. Interview Explanation

"Set bit count is the number of 1s in binary. A fast manual method is to repeatedly do $x = x \& (x - 1)$, which removes the lowest set bit each time, so the loop runs exactly once per set bit."

19. Revision Notes

- C++: `__builtin_popcountll(x)`.
- Python: `x.bit_count()`.
- Manual: `while (x) x &= x - 1`.
- `popcount(a ^ b)` gives Hamming distance.

20. Final Cheat Sheet

Need	Code
C++ long long popcount	<code>__builtin_popcountll(x)</code>
Python popcount	<code>x.bit_count()</code>
Manual count	<code>while (x) { x &= x - 1; count++; }</code>
Hamming distance	<code>popcount(a ^ b)</code>

Power of Two Check

1. Overview

A number is a power of two if its binary representation has exactly one set bit.

Examples:

```
1 = 1
2 = 10
4 = 100
8 = 1000
```

2. Intuition

For powers of two, there is only one **1** bit. Subtracting **1** turns that bit off and makes all lower bits **1**.

```
8      = 1000
7      = 0111
8 & 7 = 0000
```

So $n > 0$ and $(n \& (n - 1)) == 0$.

3. When to Use It

Use this when:

- Problem asks if a number is a power of two.
- You need validate mask with exactly one item.
- You need check whether a value has exactly one set bit.
- You are detecting lowbit-only states.

Trigger phrases:

- "power of two"
- "exactly one bit set"
- "single flag"
- "one selected element"

4. When Not to Use It

Do not use it when:

- Number can be **0**; always handle $n > 0$.
- You need power of another base, like 3 or 4.
- Floating point logs could cause precision issues; avoid logs for this.

5. Core Concepts

Concept	Explanation
One set bit	Defining property of powers of two
$n \& (n - 1)$	Clears lowest set bit
Positive check	0 is not a power of two
Power of four	Power of two plus bit position even

6. Step-by-Step Algorithm

1. If $n \leq 0$, return false.
2. Compute $n \& (n - 1)$.
3. If result is 0 , return true.
4. Otherwise, return false.

7. Dry Run

Input: $n = 16$

Step	Binary
n	10000
$n - 1$	01111
$n \& (n - 1)$	00000

Answer: true.

Input: $n = 12$

Step	Binary
n	1100
$n - 1$	1011
AND	1000

Answer: false.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

bool isPowerOfTwo(long long n) {
    return n > 0 && (n & (n - 1)) == 0;
}

int main() {
```

```

    cout << boolalpha;
    cout << isPowerOfTwo(16) << '\n';
    cout << isPowerOfTwo(12) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def is_power_of_two(n: int) -> bool:
    return n > 0 and (n & (n - 1)) == 0

print(is_power_of_two(16))
print(is_power_of_two(12))

```

10. Code Explanation

The condition `n > 0` rejects zero and negatives. The expression `n & (n - 1)` removes the lowest set bit. A power of two has only one set bit, so removing it leaves zero.

11. Complexity Analysis

Operation	Time	Space
Power of two check	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Exact one selected item	Mask has one bit	<code>(mask & (mask - 1)) == 0</code>	Bitmask DP base cases
Power of two	Direct numeric check	Same formula	LeetCode 231
Game states	Moves depend on powers of two	Check lowbit property	CP games

13. Common Mistakes

- Forgetting `n > 0`; otherwise `0` incorrectly passes.
- Using `log2` and floating precision.
- Thinking all even numbers are powers of two.
- Overflow when computing powers first.

14. Edge Cases

- `n = 0`

- $n = 1$
- Negative n
- Large power like $1LL \ll 60$
- Number with many bits set

15. Variations

Variation	Use
Power of four	Power of two and set bit at even position
Exactly one selected item	Same check on mask
Next power of two	Bit spreading technique

16. Related Algorithms/Data Structures

- Brian Kernighan uses the same $n \& (n - 1)$ idea.
- Lowest set bit gives the single bit if number is a power of two.
- Count set bits can also check $\text{popcount}(n) == 1$.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Power of Two	LeetCode	One set bit	Easy
Power of Four	LeetCode	Power of two plus position	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Count Good Numbers Variants	CP	Powers and masks	Medium
Bitwise AND of Numbers Range	LeetCode	Common power prefix	Medium
Maximum XOR Queries	LeetCode	Powers of two/trie bits	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Nim-like Bit Games	Codeforces	Powers of two states	Hard
Xor of 3	CodeChef	Bit properties	Hard
Bitmask Game DP	AtCoder	One-bit masks	Hard

18. Interview Explanation

"A positive power of two has exactly one set bit. If I compute $n \& (n - 1)$, it removes the lowest set bit. For powers of two, that leaves zero, so the check is $n > 0 \&\& (n \& (n - 1)) == 0$."

19. Revision Notes

- Formula: $n > 0 \&\& (n \& (n - 1)) == 0$.
- 0 is not a power of two.
- Avoid floating point logs.
- Same check works for one-element masks.

20. Final Cheat Sheet

Need	Code
Power of two	$n > 0 \&\& (n \& (n - 1)) == 0$
Exactly one bit	$(mask \& (mask - 1)) == 0$ with $mask > 0$
Alternative	$popcount(n) == 1$

XOR Properties

1. Overview

XOR is a bitwise operation with powerful algebraic properties. It is central to unique-number problems, prefix XOR, parity masks, and XOR basis.

2. Intuition

XOR behaves like "different gives 1, same gives 0". When the same value appears twice, it cancels out.

$$\begin{aligned} x \wedge x &= 0 \\ x \wedge 0 &= x \end{aligned}$$

This makes XOR useful for problems where pairs should disappear.

3. When to Use It

Use XOR when:

- Every duplicate appears an even number of times.
- Need parity of counts.
- Need range XOR queries.
- Need to compare different bits.
- Need maximum subset XOR or XOR basis.

Trigger phrases:

- "appears twice except one"
- "odd occurrence"
- "range XOR"
- "parity"
- "maximum XOR"

4. When Not to Use It

Do not use XOR when:

- You need sums or frequencies, not parity.
- Duplicates may appear odd counts unpredictably.
- You need ordering or min/max by value without bit logic.
- The operation is AND/OR based; XOR has different behavior.

5. Core Concepts

Property	Formula	Meaning
Identity	$x \oplus 0 = x$	Zero changes nothing
Self inverse	$x \oplus x = 0$	Equal values cancel
Commutative	$a \oplus b = b \oplus a$	Order does not matter
Associative	$(a \oplus b) \oplus c = a \oplus (b \oplus c)$	Can group freely
Toggle	$x \oplus \text{mask}$	Flips mask bits

6. Step-by-Step Algorithm

For duplicate cancellation:

1. Initialize `answer = 0`.
2. XOR every array element into `answer`.
3. All paired equal values cancel.
4. The remaining value is the unique odd-occurring value.

7. Dry Run

Input: `[4, 1, 2, 1, 2]`

Step	Value	XOR so far
Start	-	0
1	4	4
2	1	5
3	2	7
4	1	6

Step	Value	XOR so far
5	2	4

Final answer: 4

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int xorAll(const vector<int>& nums) {
    int result = 0;
    for (int value : nums) {
        result ^= value;
    }
    return result;
}

int main() {
    vector<int> nums = {4, 1, 2, 1, 2};
    cout << xorAll(nums) << '\n';
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def xor_all(nums: list[int]) -> int:
    result = 0
    for value in nums:
        result ^= value
    return result

print(xor_all([4, 1, 2, 1, 2]))
```

10. Code Explanation

`result` starts at 0 because XOR with zero leaves a number unchanged. Each value is XORed into the result. Duplicate pairs cancel due to $x \oplus x = 0$.

11. Complexity Analysis

Operation	Time	Space
XOR all elements	$O(n)$	$O(1)$
Range XOR with prefix	$O(n)$ preprocessing, $O(1)$ query	$O(n)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Single number	Pairs and one unique	XOR all	LeetCode 136
Range XOR	Many subarray XOR queries	Prefix XOR	LeetCode 1310
Hamming distance	Different bits	Popcount XOR	LeetCode 461
Parity masks	Odd/even character counts	Toggle char bit	LeetCode 1371

13. Common Mistakes

- Using XOR when values appear three times; basic XOR fails.
- Forgetting XOR is bitwise, not arithmetic addition.
- Assuming XOR preserves magnitude order.
- Confusing XOR with OR.

14. Edge Cases

- Empty array, if allowed.
- Single element.
- All pairs except one.
- Values include zero.
- Negative values.

15. Variations

Variation	Use
Prefix XOR	Range XOR queries
XOR basis	Maximum subset XOR
XOR trie	Maximum pair XOR
Parity mask	Palindrome/anagram properties

16. Related Algorithms/Data Structures

- Single number problems rely directly on XOR properties.
- XOR basis generalizes XOR independence.
- Fast Walsh-Hadamard Transform computes XOR convolution.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Single Number	LeetCode	XOR cancellation	Easy

Problem	Platform	Main idea/pattern	Difficulty
Hamming Distance	LeetCode	XOR and popcount	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
XOR Queries of a Subarray	LeetCode	Prefix XOR	Medium
Single Number III	LeetCode	XOR partition	Medium
Find Longest Awesome Substring	LeetCode	Parity mask	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Maximum Genetic Difference Query	LeetCode	XOR trie	Hard
Maximum XOR With an Element From Array	LeetCode	Offline trie	Hard
XOR Basis	Codeforces	Linear basis	Hard

18. Interview Explanation

"XOR is useful because it is associative, commutative, and each value cancels itself. So when every duplicate appears twice, XORing everything leaves only the unique value. It also naturally represents parity and bit differences."

19. Revision Notes

- $x \oplus x = 0$.
- $x \oplus 0 = x$.
- Order does not matter.
- Prefix XOR: $\text{xor}(1, r) = \text{pref}[r + 1] \oplus \text{pref}[1]$.
- For appears-three-times problems, use bit counting, not simple XOR.

20. Final Cheat Sheet

Need	Code
Unique among pairs	XOR all
Range XOR	$\text{pref}[r + 1] \oplus \text{pref}[1]$
Different bits	$a \oplus b$
Toggle mask	$x \oplus \text{mask}$

Single Number Problems

1. Overview

Single number problems ask you to find elements that appear a different number of times than the rest. They are common in interviews because they test XOR and bit counting.

2. Intuition

If every number appears twice except one, XOR all values and pairs cancel.

If every number appears three times except one, XOR is not enough. Count each bit modulo 3. Bits belonging to tripled numbers vanish modulo 3, leaving the unique number.

3. When to Use It

Use these techniques when:

- Array has duplicates with strict occurrence counts.
- Need $O(n)$ time and $O(1)$ extra space.
- Problem mentions "every element appears twice/thrice except..."
- Need find two unique numbers among pairs.

4. When Not to Use It

Do not use these tricks when:

- Occurrence counts are not fixed.
- You need all frequencies.
- Input is not integer-like.
- Simpler hash map is accepted and less error-prone.

5. Core Concepts

Case	Key idea
One unique, others twice	XOR all
Two unique, others twice	XOR all, split by a differing bit
One unique, others thrice	Count bits modulo 3
General k times	Bit count modulo k

6. Step-by-Step Algorithm

For two unique numbers:

1. XOR all elements to get $\text{xorAll} = a \oplus b$.
2. Find a set bit in xorAll ; this bit differs between a and b .
3. Partition numbers into two groups based on that bit.
4. XOR each group separately.
5. The results are the two unique numbers.

7. Dry Run

Input: [1, 2, 1, 3, 2, 5]

Step	Value
XOR all	$3 \oplus 5 = 6$
Binary 6	110
Lowest set bit	010

Partition by bit 1:

Group	Numbers	XOR
Bit not set	1, 1, 5	5
Bit set	2, 3, 2	3

Answer: 3 and 5

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int singleNumber(const vector<int>& nums) {
    int result = 0;
    for (int value : nums) result ^= value;
    return result;
}

pair<int, int> twoSingleNumbers(const vector<int>& nums) {
    int xorAll = 0;
    for (int value : nums) xorAll ^= value;

    int separatingBit = xorAll & -xorAll;
    int first = 0, second = 0;

    for (int value : nums) {
        if (value & separatingBit) first ^= value;
        else second ^= value;
    }

    return {first, second};
}

int singleNumberWhenOthersThrice(const vector<int>& nums) {
    int answer = 0;
    for (int bit = 0; bit < 32; bit++) {
        int count = 0;
        for (int value : nums) {
```

```

        if ((value >> bit) & 1) count++;
    }
    if (count % 3 != 0) {
        answer |= (1 << bit);
    }
}
return answer;
}

int main() {
    vector<int> a = {4, 1, 2, 1, 2};
    vector<int> b = {1, 2, 1, 3, 2, 5};
    vector<int> c = {2, 2, 3, 2};

    cout << singleNumber(a) << '\n';
    auto [x, y] = twoSingleNumbers(b);
    cout << x << " " << y << '\n';
    cout << singleNumberWhenOthersThrice(c) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def single_number(nums: list[int]) -> int:
    result = 0
    for value in nums:
        result ^= value
    return result

def two_single_numbers(nums: list[int]) -> tuple[int, int]:
    xor_all = 0
    for value in nums:
        xor_all ^= value

    separating_bit = xor_all & -xor_all
    first = second = 0
    for value in nums:
        if value & separating_bit:
            first ^= value
        else:
            second ^= value
    return first, second

def single_number_when_others_thrice(nums: list[int]) -> int:
    answer = 0
    for bit in range(32):
        count = sum((value >> bit) & 1 for value in nums)
        if count % 3:
            answer |= 1 << bit

```

```

if answer >= 1 << 31:
    answer -= 1 << 32
return answer

```

10. Code Explanation

`singleNumber` XORs all elements, so pairs cancel. `twoSingleNumbers` first obtains $a \oplus b$, then uses one set bit to separate the two unique numbers into different groups. `singleNumberWhenOthersThrice` counts each bit modulo 3 and reconstructs the unique number.

11. Complexity Analysis

Problem type	Time	Space
One unique, pairs	$O(n)$	$O(1)$
Two unique, pairs	$O(n)$	$O(1)$
One unique, triples	$O(32n) = O(n)$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
All twice except one	Pairs cancel	XOR all	LeetCode 136
All twice except two	Need two uniques	Split by set bit	LeetCode 260
All thrice except one	Modulo bit counts	Count bits mod 3	LeetCode 137

13. Common Mistakes

- Applying simple XOR to "appears three times" case.
- Forgetting negative number handling in Python for fixed 32-bit problems.
- Choosing a separating bit that is not set in `xorAll`.
- Returning sorted pair when problem does not require sorting, or not sorting when it does.

14. Edge Cases

- Unique number is 0.
- Unique number is negative.
- Array length is 1.
- Two unique numbers have many common bits.
- Large values near 32-bit boundary.

15. Variations

Variation	Use
k times except one	Count bits modulo k

Variation	Use
Two unique numbers	XOR partition
Streaming XOR	Unique among pairs can be processed online
State-machine bit trick	Ones/twos variables for triples

16. Related Algorithms/Data Structures

- XOR properties are the foundation.
- Lowest set bit helps separate two unique numbers.
- Bit counting solves modulo-frequency variants.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Single Number	LeetCode	XOR all	Easy
Missing Number	LeetCode	XOR index and values	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Single Number II	LeetCode	Bit count modulo 3	Medium
Single Number III	LeetCode	XOR partition	Medium
Find the Duplicate Number Variant	GFG	Bit/count logic	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Find Unique in k Occurrences	GFG	General bit modulo	Hard
Maximum XOR Pair Variants	Codeforces	XOR reasoning	Hard
XOR Basis Unique Queries	Codeforces	Linear basis	Hard

18. Interview Explanation

"For the classic single number problem, I use XOR because equal numbers cancel out and XOR is associative and commutative. For two unique numbers, I XOR all values to get $a \oplus b$, pick a set bit where they differ, partition the array by that bit, and XOR each group."

19. Revision Notes

- Pairs except one: XOR all.

- Pairs except two: split by `xorAll` & `-xorAll`.
- Triples except one: count bits modulo 3.
- Handle negative reconstruction in Python if using 32-bit logic.

20. Final Cheat Sheet

Case	Key code
One unique	<code>ans ^= x</code>
Two uniques	<code>bit = xorAll & -xorAll</code>
Others thrice	<code>count[bit] % 3</code>
Missing number	XOR range and array

Bitmasking

1. Overview

Bitmasking represents a set of items using bits of an integer. If bit `i` is `1`, item `i` is included; if it is `0`, item `i` is absent.

2. Intuition

Instead of storing `[true, false, true]`, store `101` in binary. This compactly represents selected items and lets us update/check states with fast bit operations.

3. When to Use It

Use bitmasking when:

- Number of items is small, usually `n <= 20` for enumeration.
- Need represent subsets.
- Need DP over visited/selected items.
- Need compress multiple booleans into one integer.
- Need fast set union/intersection over small universe.

Trigger phrases:

- "subset"
- "selected items"
- "visited all nodes"
- "skills covered"
- "`n <= 20`"

4. When Not to Use It

Avoid bitmasking when:

- `n` is too large for `2^n` algorithms.

- IDs are sparse or unbounded.
- You need ordered operations like predecessor/successor.
- A simple array, set, or graph traversal is clearer.

5. Core Concepts

Concept	Code	Meaning
Empty set	0	No items
Full set	$(1 \ll n) - 1$	All n items
Check item	$\text{mask} \& (1 \ll i)$	Is item present
Add item	$\text{mask} (1 \ll i)$	$(1 \ll i)$
Remove item	$\text{mask} \& \sim(1 \ll i)$	Exclude item

6. Step-by-Step Algorithm

To use a bitmask:

1. Assign each item an index from 0 to $n - 1$.
2. Start with $\text{mask} = 0$.
3. Add items using OR.
4. Check membership using AND.
5. Iterate masks from 0 to $(1 \ll n) - 1$ if all subsets are needed.

7. Dry Run

Items: A, B, C

Selected: A and C

Item	Index	Bit
A	0	1
B	1	0
C	2	1

Mask: 101 binary = 5

Check B:

$$\text{mask} \& (1 \ll 1) = 101 \& 010 = 000$$

So B is not selected.

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

struct BitMaskSet {
    int mask = 0;

    void add(int i) {
        mask |= (1 << i);
    }

    void remove(int i) {
        mask &= ~(1 << i);
    }

    bool contains(int i) const {
        return (mask & (1 << i)) != 0;
    }

    int size() const {
        return __builtin_popcount(mask);
    }
};

int main() {
    BitMaskSet s;
    s.add(0);
    s.add(2);
    cout << s.contains(1) << '\n';
    cout << s.size() << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

class BitMaskSet:
    def __init__(self) -> None:
        self.mask = 0

    def add(self, i: int) -> None:
        self.mask |= 1 << i

    def remove(self, i: int) -> None:
        self.mask &= ~(1 << i)

    def contains(self, i: int) -> bool:
        return (self.mask & (1 << i)) != 0

    def size(self) -> int:
        return self.mask.bit_count()

```

```
s = BitMaskSet()
s.add(0)
s.add(2)
print(s.contains(1))
print(s.size())
```

10. Code Explanation

The struct stores the entire set in `mask`. `add` sets a bit, `remove` clears a bit, `contains` checks a bit, and `size` uses popcount to count selected items.

11. Complexity Analysis

Operation	Time	Space
Add/remove/check	$O(1)$	$O(1)$
Count selected	$O(1)$ built-in	$O(1)$
Enumerate all masks	$O(2^n)$	Depends on algorithm

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Subset enumeration	Need all subsets	Loop masks	LeetCode Subsets
Skill coverage	Small skill count	OR masks	Smallest Sufficient Team
Visited state	Need remember visited nodes	Mask in BFS/DP	Shortest Path Visiting All Nodes

13. Common Mistakes

- Trying 2^n when $n = 40$.
- Using `pow(2, n)` instead of `1 << n`.
- Forgetting item-to-index mapping.
- Overflowing `int` masks.
- Misreading mask decimal values.

14. Edge Cases

- Empty mask.
- Full mask.
- Single item.
- Duplicate items in input.
- $n = 0$.
- n near 31 or 63.

15. Variations

Variation	Use
<code>long long</code> mask	<code>n <= 60</code> membership only
<code>std::bitset</code>	Larger fixed-size bit operations
Dynamic bitset/vector words	Very large bitsets
Submask iteration	Enumerate subsets of a mask

16. Related Algorithms/Data Structures

- Subsets using bitmask is the basic enumeration technique.
- DP with bitmask uses masks as DP states.
- Bitset optimization uses many machine words for larger masks.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Subsets	LeetCode	Enumerate masks	Easy/Medium
Maximum Product of Word Lengths	LeetCode	Character masks	Medium

Medium

Problem	Platform	Main idea/pattern	Difficulty
Beautiful Arrangement	LeetCode	Mask DP	Medium
Partition to K Equal Sum Subsets	LeetCode	Used mask	Medium
Smallest Sufficient Team	LeetCode	Skill masks	Medium/Hard

Hard

Problem	Platform	Main idea/pattern	Difficulty
Shortest Path Visiting All Nodes	LeetCode	BFS with mask	Hard
Hamiltonian Flights	CSES	Bitmask DP	Hard
Close Group	AtCoder	Subset DP	Hard

18. Interview Explanation

"Bitmasking stores a subset inside an integer. Each bit represents whether an item is selected. This gives constant-time add, remove, and membership checks, and it is especially useful when `n` is small enough for 2^n states."

19. Revision Notes

- Empty mask: 0 .
- Full mask: $(1 \ll n) - 1$.
- Check: $\text{mask} \& (1 \ll i)$.
- Add: $\text{mask} \mid (1 \ll i)$.
- Remove: $\text{mask} \& \sim(1 \ll i)$.
- Enumeration is $O(2^n)$.

20. Final Cheat Sheet

Need	Code
Empty set	0
Full set	$(1 \ll n) - 1$
Contains	$\text{mask} \& (1 \ll i)$
Add	$\text{mask} \mid (1 \ll i)$
Remove	$\text{mask} \& \sim(1 \ll i)$
Size	$\text{popcount}(\text{mask})$

Subsets Using Bitmask

1. Overview

Subsets using bitmask is a technique to generate all subsets of n items by iterating masks from 0 to $(1 \ll n) - 1$.

2. Intuition

For n items, each item has two choices: absent or present. That gives 2^n subsets. A binary mask naturally stores these choices.

For items $[a, b, c]$:

```
000 -> []
001 -> [a]
010 -> [b]
011 -> [a, b]
...
111 -> [a, b, c]
```

3. When to Use It

Use this when:

- Need generate all subsets.
- Need brute force over all choices and $n \leq 20$.
- Need check every combination.
- Need subset sums for meet-in-the-middle.

Trigger phrases:

- "all subsets"
- "all combinations"
- "choose any elements"
- "n is small"

4. When Not to Use It

Avoid it when:

- n is large, usually above 25 for full enumeration.
- Need combinations of fixed size and combinatorial generation is more efficient.
- The problem has a greedy or DP solution avoiding 2^n .

5. Core Concepts

Concept	Meaning
2^n masks	One for every subset
Bit i	Whether item i is included
Empty subset	Mask 0
Full subset	Mask $(1 \ll n) - 1$

6. Step-by-Step Algorithm

1. Let n be the number of items.
2. Loop $mask$ from 0 to $(1 \ll n) - 1$.
3. Create an empty current subset.
4. For each index i , check if bit i is set.
5. If set, add $items[i]$ to the subset.
6. Process or store the subset.

7. Dry Run

Input: $[10, 20, 30]$

Mask	Binary	Subset
0	000	$[]$
1	001	$[10]$
2	010	$[20]$

Mask	Binary	Subset
3	011	[10, 20]
4	100	[30]
5	101	[10, 30]
6	110	[20, 30]
7	111	[10, 20, 30]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> generateSubsets(const vector<int>& nums) {
    int n = nums.size();
    vector<vector<int>> subsets;

    for (int mask = 0; mask < (1 << n); mask++) {
        vector<int> current;
        for (int i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                current.push_back(nums[i]);
            }
        }
        subsets.push_back(current);
    }

    return subsets;
}

int main() {
    vector<int> nums = {10, 20, 30};
    auto subsets = generateSubsets(nums);

    for (const auto& subset : subsets) {
        cout << "{ ";
        for (int value : subset) cout << value << ' ';
        cout << "}\n";
    }
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def generate_subsets(nums: list[int]) -> list[list[int]]:
    n = len(nums)
    subsets = []
```

```

    for mask in range(1 << n):
        current = []
        for i in range(n):
            if mask & (1 << i):
                current.append(nums[i])
            subsets.append(current)
        return subsets

print(generate_subsets([10, 20, 30]))

```

10. Code Explanation

The outer loop chooses a subset mask. The inner loop checks each bit and adds the corresponding array element if the bit is set. The result contains all 2^n subsets.

11. Complexity Analysis

Task	Time	Space
Generate all subsets	$O(n * 2^n)$	$O(n * 2^n)$ if stored
Process only	$O(n * 2^n)$	$O(n)$ current subset

12. Common Patterns

Pattern	Identify it	Approach	Example problems
All combinations	Need every subset	Bitmask loop	LeetCode Subsets
Subset sum	Need all possible sums	Add selected values	Meet-in-the-middle
Fixed-size subset	Need exactly k items	Filter by popcount	CP combinatorics

13. Common Mistakes

- Using $1 \ll n$ when $n \geq 31$ with `int`.
- Forgetting empty subset.
- Assuming generated order is lexicographic.
- Storing all subsets when only a count or best value is needed.

14. Edge Cases

- Empty input, only subset is empty.
- Single element.
- Duplicate values.
- Large n .
- Negative values.

15. Variations

Variation	Use
Filter by popcount	Subsets of size k
Submask iteration	Subsets of a given mask
Meet-in-the-middle	n around 40
Recursive backtracking	More natural for duplicate handling

16. Related Algorithms/Data Structures

- Bitmasking is the representation.
- DP with bitmask uses the same mask space but stores optimal values.
- Meet-in-the-middle reduces 2^n for larger n .

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Subsets	LeetCode	Enumerate masks	Easy/Medium
Sum of All Subset XOR Totals	LeetCode	Enumerate subsets	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Subsets II	LeetCode	Duplicates	Medium
Partition Equal Subset Sum Variant	GFG	Subset search	Medium
Maximum Product of Word Lengths	LeetCode	Subset/char masks	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Meet in the Middle Subset Sum	CSES/CP	Split subsets	Hard
Minimum Incompatibility	LeetCode	Valid subset precompute	Hard
Close Group	AtCoder	Subset enumeration	Hard

18. Interview Explanation

"For n elements, every subset corresponds to an n -bit number. I iterate from 0 to $2^n - 1$; if bit i is set in the mask, I include `nums[i]` in the current subset."

19. Revision Notes

- Total subsets: 2^n .
- Loop: `for mask in [0, 1 << n).`
- Include item: `if (mask & (1 << i)).`
- Time: $O(n * 2^n)$.
- Watch out for large n .

20. Final Cheat Sheet

Need	Code
All masks	<code>for (int mask = 0; mask < (1 << n); mask++)</code>
Include item	<code>if (mask & (1 << i))</code>
Subset size	<code>__builtin_popcount(mask)</code>
Full mask	<code>(1 << n) - 1</code>

DP with Bitmask

1. Overview

DP with bitmask uses a bitmask as part of the DP state, usually to represent which items, nodes, or skills have already been used or covered.

2. Intuition

Normal DP remembers a small number of values like index or capacity. Bitmask DP remembers an entire subset. This is useful when future choices depend on exactly which items are already selected.

Example: In Traveling Salesman style DP, `dp[mask][last]` means the minimum cost to visit the set `mask` and end at `last`.

3. When to Use It

Use bitmask DP when:

- n is small, usually $n \leq 20$.
- State depends on a subset of items.
- Need optimize over all orders or selected sets.
- Problems mention visiting all nodes, assigning tasks, covering skills, or Hamiltonian paths.

Trigger phrases:

- "visit all"
- "assign each"
- "minimum cost to cover"
- " $n \leq 20$ "
- "Hamiltonian"

4. When Not to Use It

Avoid it when:

- n is too large for 2^n .
- Greedy or graph shortest path without subset state is enough.
- State can be compressed further.
- You only need simple subset enumeration, not optimal substructure.

5. Core Concepts

Concept	Meaning
mask	Selected/visited set
last	Last chosen item/node
Transition	Add one missing item
Base case	Usually one selected item or empty mask
Full mask	Target state where all items selected

6. Step-by-Step Algorithm

For TSP-style DP:

1. Define $dp[mask][last]$ as minimum cost to reach $last$ after visiting $mask$.
2. Initialize $dp[1 \ll start][start] = 0$.
3. Iterate over all masks.
4. For each possible $last$ inside mask, try adding an unvisited $next$.
5. Update $dp[mask | (1 \ll next)][next]$.
6. Answer is minimum over full-mask ending states.

7. Dry Run

Suppose distances:

```
0 -> 1 = 5
0 -> 2 = 2
1 -> 2 = 1
2 -> 1 = 4
```

Start at 0, visit all.

State	Value	Transition
$dp[001][0]$	0	Start
$dp[011][1]$	5	Add 1

State	Value	Transition
<code>dp[101][2]</code>	2	Add 2
<code>dp[111][2]</code>	6	From 1 add 2
<code>dp[111][1]</code>	6	From 2 add 1

Minimum full path cost: 6.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;

int shortestHamiltonianPath(const vector<vector<int>>& cost, int start) {
    int n = cost.size();
    int totalMasks = 1 << n;
    vector<vector<int>> dp(totalMasks, vector<int>(n, INF));

    dp[1 << start][start] = 0;

    for (int mask = 0; mask < totalMasks; mask++) {
        for (int last = 0; last < n; last++) {
            if (dp[mask][last] == INF) continue;
            if ((mask & (1 << last)) == 0) continue;

            for (int next = 0; next < n; next++) {
                if (mask & (1 << next)) continue;
                int nextMask = mask | (1 << next);
                dp[nextMask][next] = min(dp[nextMask][next],
                                         dp[mask][last] + cost[last][next]);
            }
        }
    }

    int fullMask = totalMasks - 1;
    return *min_element(dp[fullMask].begin(), dp[fullMask].end());
}

int main() {
    vector<vector<int>> cost = {
        {0, 5, 2},
        {5, 0, 1},
        {2, 4, 0}
    };
    cout << shortestHamiltonianPath(cost, 0) << '\n';
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def shortest_hamiltonian_path(cost: list[list[int]], start: int) -> int:
    n = len(cost)
    total_masks = 1 << n
    inf = 10**18
    dp = [[inf] * n for _ in range(total_masks)]
    dp[1 << start][start] = 0

    for mask in range(total_masks):
        for last in range(n):
            if dp[mask][last] == inf:
                continue
            if not (mask & (1 << last)):
                continue
            for nxt in range(n):
                if mask & (1 << nxt):
                    continue
                next_mask = mask | (1 << nxt)
                dp[next_mask][nxt] = min(
                    dp[next_mask][nxt],
                    dp[mask][last] + cost[last][nxt],
                )

    return min(dp[total_masks - 1])
```

10. Code Explanation

`dp[mask][last]` stores the best cost for a precise visited set and final node. The transition adds one unvisited node. The algorithm tries all states and all possible next choices, so it covers every valid visiting order.

11. Complexity Analysis

Task	Time	Space
TSP-style DP	$O(2^n * n^2)$	$O(2^n * n)$
Assignment DP	$O(2^n * n)$	$O(2^n)$
Skill cover DP	$O(\text{people} * 2^{\text{skills}})$	$O(2^{\text{skills}})$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Visit all nodes	Need shortest path with visited set	BFS/DP with mask	LeetCode 847
Assignment	One task per worker	<code>dp[mask]</code>	AtCoder DP O Matching
Hamiltonian paths	Count/order visits	<code>dp[mask][last]</code>	CSES Hamiltonian Flights

Pattern	Identify it	Approach	Example problems
Skill cover	Cover all skills	OR skill masks	LeetCode 1125

13. Common Mistakes

- Forgetting base cases.
- Updating from invalid states.
- Using `int` when costs can overflow.
- Confusing item index with mask value.
- Trying bitmask DP when `n` is too large.

14. Edge Cases

- `n = 1`.
- Disconnected graph or impossible transitions.
- Zero-cost edges.
- Multiple optimal answers.
- Full mask already reached.

15. Variations

Variation	Use
<code>dp[mask]</code>	Assignment or subset cost
<code>dp[mask][last]</code>	Paths/orders
BFS with mask	Unweighted shortest path
Memoized recursion	Easier for interviews
SOS DP	Aggregate over submasks faster

16. Related Algorithms/Data Structures

- Subsets using bitmask enumerates states.
- BFS with state compression handles unweighted graph versions.
- SOS DP improves subset aggregation transitions.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Beautiful Arrangement	LeetCode	Mask DP	Medium
Can I Win	LeetCode	Memo mask	Medium

Medium

Problem	Platform	Main idea/pattern	Difficulty
Partition to K Equal Sum Subsets	LeetCode	Used mask DP	Medium
Matchsticks to Square	LeetCode	Mask state	Medium
Maximum Students Taking Exam	LeetCode	Row mask DP	Medium/Hard

Hard

Problem	Platform	Main idea/pattern	Difficulty
Shortest Path Visiting All Nodes	LeetCode	BFS with mask	Hard
Hamiltonian Flights	CSES	Count paths DP	Hard
AtCoder DP O Matching	AtCoder	Assignment bitmask DP	Hard

18. Interview Explanation

"I use bitmask DP when the state depends on a subset. The mask tells me which items are already used, and the DP value stores the best answer for that exact set. Then I transition by adding one missing item."

19. Revision Notes

- $n \leq 20$ is a big hint.
- `dp[mask]` or `dp[mask][last]`.
- Full mask: $(1 \ll n) - 1$.
- TSP style time: $O(2^n * n^2)$.
- Initialize impossible states carefully.

20. Final Cheat Sheet

Need	Code idea
Add next	<code>nextMask = mask</code>
Check visited	<code>mask & (1 << i)</code>
Full target	<code>(1 << n) - 1</code>
Path DP	<code>dp[mask][last]</code>
Assignment DP	<code>dp[mask]</code>

Brian Kernighan Algorithm

1. Overview

Brian Kernighan's algorithm is a bit trick that repeatedly clears the lowest set bit of a number using `x = x & (x - 1)`.

2. Intuition

Subtracting **1** from a number flips the lowest set bit to **0** and turns all lower bits to **1**. ANDing with the original number removes that lowest set bit.

```
x      = 101100
x - 1  = 101011
AND    = 101000
```

3. When to Use It

Use it when:

- Counting set bits manually.
- Removing set bits one at a time.
- Checking power of two.
- Iterating over set bits efficiently.

Trigger phrases:

- "count 1 bits"
- "clear lowest set bit"
- "iterate set bits"
- "power of two"

4. When Not to Use It

Avoid it when:

- Built-in popcount is allowed and enough.
- Need process all bit positions including zeros.
- Number can be negative and width is unclear.

5. Core Concepts

Concept	Meaning
$x \& (x - 1)$	Clears lowest set bit
Loop count	Number of set bits
Power of two	One clearing gives zero
Sparse bits	Faster than checking every position

6. Step-by-Step Algorithm

1. Initialize **count** = **0**.
2. While **x** != **0**:
3. Set **x** = **x** & (**x** - **1**).

4. Increment `count`.

5. Return `count`.

7. Dry Run

Input: `x = 44`

44 = 101100

Iteration	x before	x after	Count
1	101100	101000	1
2	101000	100000	2
3	100000	000000	3

Answer: 3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

int kernighanCount(long long x) {
    int count = 0;
    while (x > 0) {
        x &= x - 1;
        count++;
    }
    return count;
}

bool isPowerOfTwo(long long x) {
    return x > 0 && (x & (x - 1)) == 0;
}

int main() {
    cout << kernighanCount(44) << '\n';
    cout << boolalpha << isPowerOfTwo(32) << '\n';
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def kernighan_count(x: int) -> int:
    count = 0
    while x > 0:
```

```

        x &= x - 1
        count += 1
    return count

def is_power_of_two(x: int) -> bool:
    return x > 0 and (x & (x - 1)) == 0

```

10. Code Explanation

The counting function loops only while set bits remain. Each `x &= x - 1` removes one set bit, so the number of loop iterations equals the answer.

11. Complexity Analysis

Operation	Time	Space
Count set bits	$O(\text{set bits})$	$O(1)$
Power of two check	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Count sparse bits	Few ones	Kernighan loop	LeetCode 191
Check one bit	Power of two	<code>x & (x - 1)</code>	LeetCode 231
Iterate set bits	Need selected indices	Use lowbit plus clear	CP masks

13. Common Mistakes

- Forgetting parentheses in `(x & (x - 1)) == 0`.
- Infinite loop for negative numbers.
- Using it when zero bits also matter.
- Forgetting `x > 0` for power-of-two check.

14. Edge Cases

- `x = 0`.
- `x = 1`.
- Power of two.
- All bits set.
- Negative values.

15. Variations

Variation	Use
<code>x & -x</code>	Get lowest set bit
<code>x &= x - 1</code>	Clear lowest set bit
<code>while (mask)</code>	Iterate selected elements

16. Related Algorithms/Data Structures

- Count set bits is the most common application.
- Power of two check uses the same clearing behavior.
- Fenwick tree uses lowest set bit, the companion operation.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Number of 1 Bits	LeetCode	Clear set bits	Easy
Power of Two	LeetCode	One set bit	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Counting Bits	LeetCode	Bit DP/Kernighan	Medium
Total Hamming Distance	LeetCode	Bit contribution	Medium
Sort Integers by Bits	LeetCode	Popcount	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Minimum Incompatibility	LeetCode	Popcount masks	Hard
SOS DP Tasks	Codeforces	Set bit transitions	Hard
Bitmask DP Hard Set	AtCoder	Iterate set bits	Hard

18. Interview Explanation

"Brian Kernighan's trick uses `x & (x - 1)` to remove the lowest set bit. Repeating this counts set bits in time proportional to the number of ones, which is efficient for sparse numbers."

19. Revision Notes

- `x & (x - 1)` clears lowest set bit.
- Loop count equals popcount.

- Power of two: exactly one set bit.
- Add `x > 0` check.

20. Final Cheat Sheet

Need	Code
Clear lowest set bit	<code>x &= x - 1</code>
Count set bits	<code>while (x) { x &= x - 1; cnt++; }</code>
Power of two	<code>x > 0 && (x & (x - 1)) == 0</code>

Lowest Set Bit

1. Overview

The lowest set bit is the rightmost **1** in a number's binary representation. It can be isolated using `x & -x`.

2. Intuition

In two's complement arithmetic, `-x` flips bits of `x` and adds **1**. ANDing `x` with `-x` keeps only the rightmost set bit.

```
x  = 101100
-x  = 010100
&   = 000100
```

3. When to Use It

Use it when:

- Need extract the rightmost selected item.
- Need Fenwick tree updates/queries.
- Need split XOR groups.
- Need iterate over set bits.

Trigger phrases:

- "rightmost set bit"
- "lowbit"
- "least significant 1"
- "Fenwick tree"

4. When Not to Use It

Avoid it when:

- `x = 0`; there is no set bit.

- You need highest set bit instead.
- Signed behavior is unclear in a language/problem setting.

5. Core Concepts

Concept	Code	Meaning
Lowbit	<code>x & -x</code>	Isolates lowest set bit
Clear lowbit	<code>x -= x & -x</code>	Remove it
Index of lowbit	<code>ctz(x)</code>	Position of rightmost one

6. Step-by-Step Algorithm

1. Ensure `x != 0`.
2. Compute `low = x & -x`.
3. `low` is a power of two representing the lowest set bit.
4. Use `low` directly or convert it to an index.

7. Dry Run

Input: `x = 40`

```
40 = 101000
-40 isolates through two's complement
40 & -40 = 001000 = 8
```

Lowest set bit value: `8`, index: `3`.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

long long lowestSetBitValue(long long x) {
    return x & -x;
}

int lowestSetBitIndex(long long x) {
    return __builtin_ctzll(x);
}

int main() {
    long long x = 40;
    cout << lowestSetBitValue(x) << '\n';
    cout << lowestSetBitIndex(x) << '\n';
}
```

```
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def lowest_set_bit_value(x: int) -> int:
    return x & -x

def lowest_set_bit_index(x: int) -> int:
    low = x & -x
    return low.bit_length() - 1

print(lowest_set_bit_value(40))
print(lowest_set_bit_index(40))
```

10. Code Explanation

$x \& -x$ returns a number with only the rightmost set bit preserved. `__builtin_ctzll` counts trailing zeros, which is exactly the index of the lowest set bit. Do not call `ctz` with zero.

11. Complexity Analysis

Operation	Time	Space
Lowbit value	$O(1)$	$O(1)$
Lowbit index	$O(1)$	$O(1)$
Iterate all set bits	$O(\text{set bits})$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Fenwick tree	Move by lowbit	<code>idx += idx & -idx</code>	Range sum queries
XOR split	Need differing bit	<code>xorAll & -xorAll</code>	Single Number III
Iterate selected items	Need indexes in mask	Extract lowbit repeatedly	Bitmask DP

13. Common Mistakes

- Calling `__builtin_ctzll(0)`, which is undefined.
- Confusing lowbit value and lowbit index.
- Using `x & (x - 1)` when you need to isolate, not clear.
- Forgetting masks are zero-indexed.

14. Edge Cases

- $x = 0$.
- $x = 1$.
- x is a power of two.
- x has many low zeros.
- Large 64-bit values.

15. Variations

Variation	Use
$x \& -x$	Lowbit value
<code>__builtin_ctzll(x)</code>	Lowbit index
$x \&= x - 1$	Clear lowbit
Highest set bit	$63 - \text{__builtin_clzll}(x)$

16. Related Algorithms/Data Structures

- Fenwick tree is the most famous lowbit data structure.
- Brian Kernighan clears the same bit that lowbit isolates.
- Single Number III uses lowbit to split values.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Single Number III	LeetCode	Lowbit split	Medium
Power of Two	LeetCode	One lowbit	Easy

Medium

Problem	Platform	Main idea/pattern	Difficulty
Range Sum Query Mutable	LeetCode	Fenwick lowbit	Medium
Count Inversions	GFG	Fenwick tree	Medium
Kth One Queries	Codeforces	Fenwick lowbit idea	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Dynamic Range Sum Queries	CSes	Fenwick/segment tree	Hard
XOR Basis with Reconstruction	Codeforces	Bit indexes	Hard

Problem	Platform	Main idea/pattern	Difficulty
Order Statistics with BIT	SPOJ	Lowbit binary lifting	Hard

18. Interview Explanation

"The lowest set bit can be isolated with $x \& -x$. It gives a power of two containing only the rightmost 1. This is useful in Fenwick trees, set-bit iteration, and splitting numbers by a differing bit."

19. Revision Notes

- Lowbit value: $x \& -x$.
- Lowbit index: `__builtin_ctzll(x)`.
- Never call `ctz(0)`.
- Clear lowbit: $x \&= x - 1$.

20. Final Cheat Sheet

Need	Code
Lowest set bit value	$x \& -x$
Lowest set bit index	<code>__builtin_ctzll(x)</code>
Clear lowest set bit	$x \&= x - 1$
Fenwick next	$i += i \& -i$

XOR Basis Basics

1. Overview

An XOR basis, also called a linear basis over GF(2), stores a compact set of numbers that can generate all possible XOR combinations of an array.

2. Intuition

Think of numbers as binary vectors. Some numbers are redundant because they can be made by XORing others. XOR basis keeps only independent numbers, usually one representative per highest set bit.

Main use: find maximum possible subset XOR.

3. When to Use It

Use XOR basis when:

- Need maximum subset XOR.
- Need check whether a value can be represented as XOR of subset.
- Need count distinct XOR values.
- Need answer XOR independence queries.

Trigger phrases:

- "maximum subset XOR"
- "linear basis"
- "can form XOR"
- "number of distinct XORs"

4. When Not to Use It

Avoid it when:

- Problem only asks maximum XOR pair; trie may be better.
- Need subset sum with addition, not XOR.
- Need actual subset and implementation complexity is not justified.
- Values are tiny and brute force is accepted.

5. Core Concepts

Concept	Meaning
Basis vector	Independent number kept in basis
Leading bit	Highest set bit of a vector
Reduction	XOR with basis to reduce a number
Rank	Number of basis vectors
Maximum XOR	Greedy improve answer from high bits

6. Step-by-Step Algorithm

To insert x :

1. Iterate bits from high to low.
2. If bit is not set in x , continue.
3. If no basis vector exists for this bit, store x there and stop.
4. Otherwise, XOR x with that basis vector to reduce it.
5. If x becomes zero, it was dependent.

To get maximum XOR:

1. Start $answer = 0$.
2. Traverse basis from high bit to low bit.
3. If $answer \oplus basis[bit]$ is larger, update answer.
4. Return answer.

7. Dry Run

Input: $[3, 10, 5]$

```
3   = 0011
10  = 1010
5   = 0101
```

Insert	Action
3	Store at bit 1
10	Store at bit 3
5	Store at bit 2

Maximum:

Current	Try	Result
0	10	10
10	5	15
15	3	15

Answer: 15

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

struct XorBasis {
    static const int LOG = 60;
    long long basis[LOG + 1]{};

    void insertVector(long long x) {
        for (int bit = LOG; bit >= 0; bit--) {
            if ((x >> bit) & 1LL) == 0) continue;

            if (basis[bit] == 0) {
                basis[bit] = x;
                return;
            }

            x ^= basis[bit];
        }
    }

    long long getMaxXor() const {
        long long answer = 0;
        for (int bit = LOG; bit >= 0; bit--) {
            answer = max(answer, answer ^ basis[bit]);
        }
    }
};
```

```

        return answer;
    }

    bool canRepresent(long long x) const {
        for (int bit = LOG; bit >= 0; bit--) {
            if (((x >> bit) & 1LL) == 0) continue;
            if (basis[bit] == 0) return false;
            x ^= basis[bit];
        }
        return true;
    }
};

int main() {
    vector<long long> nums = {3, 10, 5};
    XorBasis xb;
    for (long long x : nums) xb.insertVector(x);
    cout << xb.getMaxXor() << '\n';
    cout << boolalpha << xb.canRepresent(15) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

class XorBasis:
    def __init__(self, log: int = 60) -> None:
        self.log = log
        self.basis = [0] * (log + 1)

    def insert(self, x: int) -> None:
        for bit in range(self.log, -1, -1):
            if not ((x >> bit) & 1):
                continue
            if self.basis[bit] == 0:
                self.basis[bit] = x
                return
            x ^= self.basis[bit]

    def max_xor(self) -> int:
        answer = 0
        for bit in range(self.log, -1, -1):
            answer = max(answer, answer ^ self.basis[bit])
        return answer

    def can_represent(self, x: int) -> bool:
        for bit in range(self.log, -1, -1):
            if not ((x >> bit) & 1):
                continue
            if self.basis[bit] == 0:
                return False

```

```

        x ^= self.basis[bit]
    return True

```

10. Code Explanation

The basis array stores at most one vector for each leading bit. Insertion reduces the incoming number using existing vectors. If it cannot be reduced to zero, it contributes new information and is stored. Maximum XOR is built greedily by trying to improve high bits first.

11. Complexity Analysis

Operation	Time	Space
Insert one number	$O(\text{LOG})$	$O(\text{LOG})$ total basis
Build basis for n numbers	$O(n * \text{LOG})$	$O(\text{LOG})$
Maximum XOR query	$O(\text{LOG})$	$O(1)$ extra
Representation check	$O(\text{LOG})$	$O(1)$ extra

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Max subset XOR	Any subset allowed	XOR basis	GFG Maximum Subset XOR
Can form value	Query target XOR	Reduce target	Codeforces basis tasks
Count distinct XORs	Need number of possible XORs	2^{rank}	CP linear basis

13. Common Mistakes

- Confusing maximum subset XOR with maximum pair XOR.
- Iterating bits low to high for basis insertion.
- Not using `long long` for large values.
- Forgetting zero vectors are dependent.
- Assuming basis stores original subset directly.

14. Edge Cases

- All numbers zero.
- Duplicate numbers.
- One number.
- Large values up to $1e18$.
- Target 0 in representation queries.

15. Variations

Variation	Use
Minimum XOR representation	Reduced row echelon form
Kth smallest XOR	Canonical basis
Basis with rollback	Offline queries
Basis on tree paths	LCA plus basis merge

16. Related Algorithms/Data Structures

- XOR properties are the foundation.
- Binary trie solves maximum XOR pair/query problems.
- Gaussian elimination is the mathematical cousin.
- FWHT handles XOR convolution, not independence.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Maximum Subset XOR	GFG	Basic XOR basis	Medium
Find Maximum XOR Basis Demo	HackerEarth	Insert/reduce	Medium

Medium

Problem	Platform	Main idea/pattern	Difficulty
XOR Basis	Codeforces EDU-style	Linear basis	Medium
Vasya and Binary String Variants	Codeforces	XOR independence	Medium
Subset XOR Queries	SPOJ	Basis query	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Xor of 3	CodeChef	Basis reasoning	Hard
Tree Path XOR Basis	Codeforces	Basis on paths	Hard
Dynamic XOR Basis	Codeforces	Rollback basis	Hard

18. Interview Explanation

"An XOR basis stores independent binary vectors. While inserting a number, I reduce it using existing basis vectors with the same high bit. If it becomes zero, it was redundant; otherwise it becomes a new basis vector. For maximum subset XOR, I greedily XOR basis vectors when they increase the answer."

19. Revision Notes

- Basis indexed by highest set bit.
- Insert from high to low.
- Maximum XOR: greedily improve answer.
- Rank = number of non-zero basis vectors.
- Distinct XORs = 2^{rank} .

20. Final Cheat Sheet

Need	Code idea
Insert	Reduce by high-bit basis
Max subset XOR	<code>ans = max(ans, ans ^ basis[i])</code>
Can represent	Reduce target to zero
Count distinct XORs	<code>1LL << rank</code>

SOS DP

1. Overview

SOS DP stands for Sum Over Subsets Dynamic Programming. It efficiently computes values over all submasks or supermasks of every mask.

Typical task: Given `f[mask]`, compute:

```
g[mask] = sum of f[submask] for all submask subset of mask
```

2. Intuition

Naively, for every mask you can iterate all its submasks. That can be $O(3^n)$. SOS DP improves this by building answers bit by bit, like a multidimensional prefix sum over bits.

3. When to Use It

Use SOS DP when:

- Need sum/min/max/count over all submasks for every mask.
- Need fast subset convolution-like preprocessing.
- `n` is around 20 and $O(n * 2^n)$ is acceptable.

Trigger phrases:

- "for every mask, over all submasks"
- "subset of mask"
- "supermask"

- "bitmask DP optimization"

4. When Not to Use It

Avoid it when:

- You need the answer for only one mask; direct submask iteration may be enough.
- The combine operation is not compatible with repeated accumulation.
- n is too large for 2^n memory.
- You need full subset convolution; SOS alone may not be enough.

5. Core Concepts

Concept	Meaning
Submask	sub where $(\text{sub} \& \text{mask}) == \text{sub}$
Supermask	sup where $(\text{sup} \& \text{mask}) == \text{mask}$
Zeta transform	Accumulate over subsets
Mobius inversion	Reverse SOS accumulation
Dimension	Each bit acts like one dimension

6. Step-by-Step Algorithm

For subset sums:

1. Copy $\text{dp} = \text{f}$.
2. For each bit from 0 to $n - 1$:
3. For each mask from 0 to $(1 \ll n) - 1$:
4. If bit is set in mask, add $\text{dp}[\text{mask} \wedge (1 \ll \text{bit})]$ to $\text{dp}[\text{mask}]$.
5. After all bits, $\text{dp}[\text{mask}]$ stores sum over all submasks.

7. Dry Run

Let $n = 2$, $\text{f}[0] = 1$, $\text{f}[1] = 2$, $\text{f}[2] = 3$, $\text{f}[3] = 4$.

Goal: $\text{g}[3] = \text{f}[0] + \text{f}[1] + \text{f}[2] + \text{f}[3] = 10$.

Stage	dp[0]	dp[1]	dp[2]	dp[3]
Initial	1	2	3	4
After bit 0	1	3	3	7
After bit 1	1	3	4	10

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

vector<long long> sumOverSubsets(vector<long long> f, int n) {
    vector<long long> dp = f;

    for (int bit = 0; bit < n; bit++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if (mask & (1 << bit)) {
                dp[mask] += dp[mask ^ (1 << bit)];
            }
        }
    }

    return dp;
}

int main() {
    int n = 2;
    vector<long long> f = {1, 2, 3, 4};
    auto g = sumOverSubsets(f, n);
    for (long long value : g) cout << value << ' ';
    cout << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def sum_over_subsets(f: list[int], n: int) -> list[int]:
    dp = f[:]
    for bit in range(n):
        for mask in range(1 << n):
            if mask & (1 << bit):
                dp[mask] += dp[mask ^ (1 << bit)]
    return dp

print(sum_over_subsets([1, 2, 3, 4], 2))

```

10. Code Explanation

The DP starts as the original array. Each bit pass adds contributions from masks that differ only by removing that bit. After processing all bits, every submask contribution has been included exactly once.

11. Complexity Analysis

Task	Time	Space
SOS over subsets	$O(n * 2^n)$	$O(2^n)$
Naive all masks and submasks	$O(3^n)$	$O(2^n)$
One-mask submask iteration	$O(2^k)$	$O(1)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Sum over submasks	sub subset mask for all masks	SOS zeta	Codeforces SOS DP
Count compatible masks	Need disjoint/contained counts	Transform complement masks	CF Compatible Numbers
Superset sums	Need all supermasks	Reverse condition	CP subset DP

13. Common Mistakes

- Updating the wrong direction for subset vs superset.
- Using SOS when only one mask query exists.
- Forgetting array size must be $1 \ll n$.
- Integer overflow in sums.
- Mixing $\text{mask} \wedge \text{bit}$ and $\text{mask} \mid \text{bit}$ incorrectly.

14. Edge Cases

- $n = 0$.
- All zeros.
- Negative values.
- Large sums requiring `long long`.
- Full mask and empty mask.

15. Variations

Variation	Use
Superset SOS	Accumulate over masks containing current mask
Mobius inversion	Recover original values
Max/min over subsets	Replace sum with max/min
AND/OR transforms	Related zeta transforms

16. Related Algorithms/Data Structures

- DP with bitmask gives the state space.
- FWHT is used for XOR convolution; SOS is for subset/superset accumulation.

- Submask iteration is the simpler one-query alternative.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Sum Over Subsets Tutorial Tasks	Codeforces EDU	Basic SOS	Medium
Count Submasks	AtCoder	Submask understanding	Medium

Medium

Problem	Platform	Main idea/pattern	Difficulty
Compatible Numbers	Codeforces	Complement mask SOS	Medium/Hard
Vowels Pair Masks	CP	Count subset-compatible strings	Medium
Subset Frequency Queries	GFG	SOS preprocessing	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
Divisibility by Subset Masks	Codeforces	SOS transform	Hard
OR Convolution Tasks	AtCoder	Zeta/Mobius	Hard
Subset Convolution	AtCoder Library Practice	Advanced subset DP	Hard

18. Interview Explanation

"SOS DP precomputes aggregate values over all submasks for every mask in $O(n * 2^n)$. It processes one bit dimension at a time, adding the contribution of masks with that bit removed."

19. Revision Notes

- Use when every mask needs all submask sums.
- Formula: if bit set, $dp[mask] += dp[mask \wedge (1 \ll bit)]$.
- Complexity: $O(n * 2^n)$.
- Naive all masks plus submasks is $O(3^n)$.

20. Final Cheat Sheet

Need	Code idea
Subset sum SOS	<code>if (mask & bit) dp[mask] += dp[mask ^ bit]</code>
Superset SOS	<code>if (!(mask & bit)) dp[mask] += dp[mask]</code>
Complexity	$O(n * 2^n)$

Need	Code idea
Memory	$O(2^n)$

Fast Walsh-Hadamard Transform

1. Overview

Fast Walsh-Hadamard Transform, or FWHT, is used to compute XOR convolution efficiently.

Given arrays **A** and **B**, XOR convolution is:

$$C[k] = \sum A[i] * B[j] \text{ where } i \oplus j = k$$

FWHT computes this in $O(n \log n)$ where **n** is a power of two.

2. Intuition

Normal convolution combines indices by addition. XOR convolution combines indices by XOR. FWHT transforms arrays into a space where XOR convolution becomes pointwise multiplication, then transforms back.

It is similar in spirit to FFT, but for XOR instead of addition.

3. When to Use It

Use FWHT when:

- Need XOR convolution.
- Need count pairs/triples where XOR has a value.
- Array size is a power of two or can be padded.
- Constraints make $O(n^2)$ too slow.

Trigger phrases:

- "XOR convolution"
- "number of pairs with xor k"
- "combine by XOR"
- "Walsh Hadamard"

4. When Not to Use It

Avoid FWHT when:

- You only need max XOR; use trie or basis.
- You need AND/OR convolution; use zeta transforms instead.
- **n** is small enough for brute force.
- Modulo division by 2 is not valid and you do not know how to invert.

5. Core Concepts

Concept	Meaning
XOR convolution	Combine indices where $i \oplus j = k$
Transform	Converts convolution to pointwise multiplication
Inverse transform	Same structure, divide by n
Butterfly	Pair update $(x + y, x - y)$
Mod inverse	Needed for modular inverse transform

6. Step-by-Step Algorithm

1. Pad arrays to power-of-two size.
2. Apply FWHT to A .
3. Apply FWHT to B .
4. Multiply pointwise: $A[i] \text{ *= } B[i]$.
5. Apply inverse FWHT.
6. Divide all values by n or multiply by modular inverse of n .
7. Result is XOR convolution.

7. Dry Run

Input:

```
A = [1, 2]
B = [3, 4]
```

Naive:

i	j	$i \oplus j$	Contribution
0	0	0	3
0	1	1	4
1	0	1	6
1	1	0	8

Result:

```
C[0] = 11
C[1] = 10
```

FWHT produces the same result in transform form.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

void fwht(vector<long long>& a, bool inverse) {
    int n = a.size();
    for (int len = 1; 2 * len <= n; len <= 1) {
        for (int i = 0; i < n; i += 2 * len) {
            for (int j = 0; j < len; j++) {
                long long x = a[i + j];
                long long y = a[i + j + len];
                a[i + j] = x + y;
                a[i + j + len] = x - y;
            }
        }
    }

    if (inverse) {
        for (long long& value : a) {
            value /= n;
        }
    }
}

vector<long long> xorConvolution(vector<long long> a, vector<long long> b) {
    int n = 1;
    while (n < (int)max(a.size(), b.size())) n <= 1;
    a.resize(n);
    b.resize(n);

    fwht(a, false);
    fwht(b, false);

    for (int i = 0; i < n; i++) {
        a[i] *= b[i];
    }

    fwht(a, true);
    return a;
}

int main() {
    vector<long long> a = {1, 2};
    vector<long long> b = {3, 4};
    auto c = xorConvolution(a, b);
    for (long long value : c) cout << value << ' ';
    cout << '\n';
    return 0;
}
```

pYTHON IMPLEMENTATION

```
def fwht(a: list[int], inverse: bool) -> None:
    n = len(a)
    length = 1
    while 2 * length <= n:
        for i in range(0, n, 2 * length):
            for j in range(length):
                x = a[i + j]
                y = a[i + j + length]
                a[i + j] = x + y
                a[i + j + length] = x - y
            length <= 1

    if inverse:
        for i in range(n):
            a[i] //= n

def xor_convolution(a: list[int], b: list[int]) -> list[int]:
    n = 1
    while n < max(len(a), len(b)):
        n <= 1
    a = a + [0] * (n - len(a))
    b = b + [0] * (n - len(b))

    fwht(a, False)
    fwht(b, False)
    for i in range(n):
        a[i] *= b[i]
    fwht(a, True)
    return a

print(xor_convolution([1, 2], [3, 4]))
```

10. Code Explanation

The butterfly step combines pairs of blocks. For each pair (x, y) , it stores $(x + y, x - y)$. After transforming both arrays, XOR convolution becomes simple pointwise multiplication. The inverse transform has the same butterfly, followed by division by n .

11. Complexity Analysis

Task	Time	Space
One FWHT	$O(n \log n)$	$O(1)$ extra
XOR convolution	$O(n \log n)$	$O(n)$ for arrays
Naive XOR convolution	$O(n^2)$	$O(n)$

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Pair XOR counts	Need count by XOR value	Frequency arrays + FWHT	CP XOR convolution
XOR polynomial product	Combine masks by XOR	FWHT	AtCoder convolution tasks
Multiple independent choices	XOR result distribution	Repeated convolution	Codeforces hard DP

13. Common Mistakes

- Forgetting size must be power of two.
- Forgetting inverse division by n .
- Using integer division when values are modulo without inverse.
- Confusing XOR convolution with normal convolution.
- Overflowing `long long` in multiplication.

14. Edge Cases

- Arrays of size `1`.
- All zeros.
- Negative values.
- Large counts requiring modulo.
- Non-power-of-two input size.

15. Variations

Variation	Use
Modular FWHT	Avoid overflow and support mod answers
AND convolution	Different transform
OR convolution	Different transform
Repeated XOR convolution	Exponentiation in transform domain

16. Related Algorithms/Data Structures

- SOS DP handles subset sums, not XOR convolution.
- FFT handles addition convolution.
- XOR basis handles independence/max subset XOR, not convolution.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
XOR Convolution Tutorial	AtCoder Library Practice	FWHT basics	Medium
Pair XOR Frequency	GFG	Frequency convolution	Medium

Medium

Problem	Platform	Main idea/pattern	Difficulty
AtCoder XOR Convolution	AtCoder	FWHT	Medium/Hard
Codeforces XOR Pair Counts	Codeforces	FWHT counts	Medium/Hard
HackerRank XOR Matrix Variants	HackerRank	Transform thinking	Medium

Hard

Problem	Platform	Main idea/pattern	Difficulty
XOR Convolution Advanced	AtCoder Library Practice	FWHT modulo	Hard
Mahmoud and a xor trip	Codeforces	XOR DP/transform	Hard
Subset XOR Distribution	Codeforces	Repeated FWHT	Hard

18. Interview Explanation

"FWHT is used for XOR convolution. It transforms arrays so that combining indices by XOR becomes pointwise multiplication, then an inverse transform recovers the result. It reduces convolution from $O(n^2)$ to $O(n \log n)$."

19. Revision Notes

- Butterfly: $(x, y) \rightarrow (x + y, x - y)$.
- Inverse is same transform plus divide by n .
- Size must be power of two.
- Use modulo inverse when working under mod.

20. Final Cheat Sheet

Need	Code idea
XOR convolution	FWHT both arrays, multiply, inverse FWHT
Butterfly	$u = x + y, v = x - y$
Complexity	$O(n \log n)$
Inverse	Divide by n

Bitset Optimization

1. Overview

Bitset optimization uses CPU word-level bit operations to speed up boolean DP, set operations, and graph-like reachability. In C++, `std::bitset<N>` stores many booleans compactly and performs operations like AND, OR, XOR, and shifts very fast.

2. Intuition

Instead of updating boolean states one by one, pack many states into machine words. One operation then updates 64 or more states internally.

Example subset sum:

```
bits[s] = whether sum s is possible
bits |= bits << value
```

This updates all sums at once.

3. When to Use It

Use bitset optimization when:

- DP state is boolean.
- Transitions are shifts, ORs, ANDs, or intersections.
- Need subset sum feasibility.
- Need fast set intersection/count.
- Constraints are too high for ordinary $O(n * \text{sum})$ but bitset can pass.

Trigger phrases:

- "possible sums"
- "boolean DP"
- "large constraints"
- "intersection count"
- "optimize with bitset"

4. When Not to Use It

Avoid it when:

- DP stores values, not booleans.
- Maximum size is not known at compile time and `std::bitset` is inconvenient.
- Need frequent dynamic resizing.
- Python performance with big integers is enough or C++ bitset is unavailable.

5. Core Concepts

Concept	Meaning
<code>bitset<N></code>	Fixed-size compact boolean array
Shift	Moves possible states
OR merge	Adds new reachable states
AND intersection	Common active states
Count	Number of set bits

6. Step-by-Step Algorithm

Subset sum feasibility:

1. Create `bitset dp`.
2. Set `dp[0] = 1`, meaning sum `0` is possible.
3. For each value `x`:
4. Shift current possible sums left by `x`.
5. OR the shifted result into `dp`.
6. At the end, `dp[target]` tells whether target sum is possible.

7. Dry Run

Values: `[2, 3]`

Initial:

```
possible sums: {0}
```

After `2`:

```
dp |= dp << 2 -> {0, 2}
```

After `3`:

```
dp << 3 gives {3, 5}
dp becomes {0, 2, 3, 5}
```

Target `5` is possible.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

const int MAX_SUM = 100000;

bool canMakeSum(const vector<int>& nums, int target) {
    bitset<MAX_SUM + 1> dp;
    dp[0] = 1;

    for (int value : nums) {
        dp |= (dp << value);
    }

    return dp[target];
}

int countCommonItems(const bitset<64>& a, const bitset<64>& b) {
    return (a & b).count();
}

int main() {
    vector<int> nums = {2, 3, 7};
    cout << boolalpha << canMakeSum(nums, 5) << '\n';

    bitset<64> a(string("1011"));
    bitset<64> b(string("1101"));
    cout << countCommonItems(a, b) << '\n';
    return 0;
}

```

pYTHON IMPLEMENTATION

```

def can_make_sum(nums: list[int], target: int) -> bool:
    bits = 1 # bit 0 is set
    for value in nums:
        bits |= bits << value
    return ((bits >> target) & 1) == 1

def count_common_items(mask_a: int, mask_b: int) -> int:
    return (mask_a & mask_b).bit_count()

print(can_make_sum([2, 3, 7], 5))
print(count_common_items(0b1011, 0b1101))

```

10. Code Explanation

`dp[0] = 1` means sum zero is possible before choosing elements. For each value, `dp << value` shifts every currently possible sum to a new sum that includes that value. OR merges old and new sums. The Python version uses a big integer as a dynamic bitset.

11. Complexity Analysis

Task	Time	Space
Normal subset sum DP	$O(n * S)$	$O(S)$
Bitset subset sum	$O(n * S / \text{word_size})$	$O(S / \text{word_size})$
Bitset AND/OR/XOR	$O(N / \text{word_size})$	$O(N / \text{word_size})$
Count set bits	$O(N / \text{word_size})$	$O(1)$ extra

12. Common Patterns

Pattern	Identify it	Approach	Example problems
Subset sum possible	Boolean sums	`bits`	<code>= bits < x</code>
Knapsack feasibility	Only possible/not possible	Bitset DP	CP subset sum
Graph common neighbors	Need intersections	<code>(adj[u] & adj[v]).count()</code>	Triangle counting
String matching variants	Bit-parallel states	Bit masks	Advanced CP

13. Common Mistakes

- Using `std::bitset` when size is only known at runtime.
- Forgetting `MAX_SUM` must cover all targets.
- Shifting by negative values.
- Using bitset DP for value optimization instead of boolean feasibility.
- Printing bitset without understanding reversed visual order.

14. Edge Cases

- Target 0.
- Empty array.
- Value 0.
- Sum greater than `MAX_SUM`.
- Large values causing shifts beyond bitset size.
- Duplicate values.

15. Variations

Variation	Use
Python integer bitset	Dynamic and concise
<code>boost::dynamic_bitset</code>	Runtime size in C++

Variation	Use
Vector of <code>uint64_t</code>	Custom high-performance bitset
Bitset graph adjacency	Fast intersections

16. Related Algorithms/Data Structures

- Bitmasking uses one integer for small sets.
- Bitset optimization scales the same idea to many bits.
- Boolean knapsack DP is often optimized by bitset shifts.
- SOS DP works over mask arrays, while bitset optimization packs booleans.

17. Practice Problems

Easy

Problem	Platform	Main idea/pattern	Difficulty
Money Sums	CSES	Bitset subset sum	Medium
Partition Equal Subset Sum	LeetCode	Bitset feasibility	Medium

Medium

Problem	Platform	Main idea/pattern	Difficulty
Knapsack Feasibility	AtCoder	Shift OR DP	Medium
Maximum Students Taking Exam	LeetCode	Row masks/bitsets	Medium
Triangle Counting	Codeforces	Bitset intersections	Medium/Hard

Hard

Problem	Platform	Main idea/pattern	Difficulty
Dynamic Reachability Bitset	Codeforces	Bitset transitive closure	Hard
Dense Graph Clique Variants	Codeforces	Bitset intersections	Hard
Large Subset Sum Queries	ICPC/CP	Bitset optimization	Hard

18. Interview Explanation

"Bitset optimization packs boolean states into machine words, so operations like OR, AND, XOR, and shifts update many states at once. For subset sum, `dp |= dp << x` adds all sums formed by including `x`, making the DP much faster in practice."

19. Revision Notes

- Use for boolean DP.

- Subset sum template: `dp[0] = 1; dp |= dp << x.`
- Complexity improves by word size factor.
- C++ `bitset<N>` needs compile-time size.
- Python integers can act like dynamic bitsets.

20. Final Cheat Sheet

Need	Code
Subset sum update	<code>`dp</code>
Test possible sum	<code>dp[target]</code>
Common elements	<code>(a & b).count()</code>
Union	<code>`a</code>
Difference-like clear	<code>a & ~b</code>